

SwimAnalyzer AI

Engineering Playbook Version 1.0

Confidential Engineering Documentation

Prepared By

System Architecture & AI Engineering Team

Project SwimAnalyzer AI

Version 1.0

Status ### In Development

Executive Summary

SwimAnalyzer AI is an AI-powered biomechanical swimming analysis platform designed to provide coaches, athletes, sports academies, and swimming clubs with objective scientific feedback using computer vision, biomechanics, and artificial intelligence.

The long-term vision extends beyond a single analysis tool. The objective is to establish a complete cloud platform that becomes the reference standard for swimming performance analysis, athlete progression tracking, and AI-assisted coaching.

The system is engineered with modular architecture, enabling gradual expansion toward:

Multi-stroke analysis Athlete management Historical performance tracking AI coaching Team management Cloud synchronization Mobile applications Enterprise deployment Vision Mission

To make professional biomechanical swimming analysis available to every swimmer in the world using Artificial Intelligence.

Vision Statement

Become the world's most trusted AI platform for swimming performance analysis.

Long-Term Objectives

The project aims to provide:

Scientific swimming analysis Automated movement evaluation Performance tracking over time AI-generated coaching recommendations Objective scoring system Training

optimization Injury prevention indicators Professional reporting ### Core Principles

The project follows six engineering principles.

Scientific First

Every metric must be scientifically explainable.

No score may exist without a measurable biomechanical reason.

Explainable AI

The AI must never produce unexplained conclusions.

Every recommendation should reference measurable evidence.

Modular Architecture

Every subsystem must remain independent.

Examples:

Video Processing

Pose Detection ### Stroke Detection Biomechanics Reports Database AI Coach

Each module can evolve independently.

Production Quality

Every feature should be production-ready before expansion.

Stability is more important than feature count.

Performance

Large swimming videos should process efficiently.

Optimization goals include:

Single-pass processing **GPU** acceleration Memory safety Streaming architecture ##

Extensibility

Future swimming styles should require minimal code changes.

The architecture must support:

Freestyle Breaststroke Butterfly Backstroke

using the same framework.

Project Scope

Current Scope

✓ Freestyle Analysis

Current capabilities include:

Pose Detection

**Stroke Cycle Detection ### Stroke Rate ### Stroke Length ### Body Roll
Joint Angles ### Consistency Validation ### Video Quality
Assessment ### Report Generation ### Athlete Database ### Historical
Charts**

Future Scope

The roadmap expands toward:

Four swimming styles AI Coach ### Training Plans ### Team Dashboard ### Club
Management ### Cloud Storage ### Mobile Apps ### Online Portal ### International
Competitions Support ### Success Criteria

The project will be considered complete when it satisfies the following objectives:

Technical

✓ Stable

✓ Fast

✓ Accurate

✓ Maintainable

✓ Tested

Scientific

- ✓ Validated
- ✓ Repeatable
- ✓ Explainable

Business

- ✓ Multi-user
- ✓ Subscription Ready
- ✓ Cloud Ready
- ✓ Enterprise Ready

Current Progress (August **2026**) Completed ### Core Engine

- ✓ Video Processing
- ✓ Pose Detection
- ✓ Stroke Detection
- ✓ Biomechanics Engine
- ✓ Video Quality Assessment
- ✓ Scientific Consistency Validator

Reliability

- ✓ Crash-Free Architecture
- ✓ Stable Streamlit Server
- ✓ Session Persistence
- ✓ Logging System
- ✓ Browser Compatible Video Export

Database

- ✓ SQLite Athlete Database
- ✓ Athlete Profiles
- ✓ Analysis History

Visualization

- ✓ Plotly Charts
- ✓ Historical Progress
- ✓ Athlete Dashboard

Reports

- ✓ **JSON** Reports
- ✓ Metrics Dashboard
- ✓ Download Center

Architecture

- ✓ Service Layer
- ✓ Modular Components
- ✓ Configuration System
- ✓ Central Logging

Remaining Roadmap

Phase 1

- ✓ Complete

Core Analysis Engine

Phase 2

- ✓ Complete

Database Integration

Phase 3

- 🔄 In Progress

Historical Performance System

Comparison Engine

Coach Dashboard

Phase 4

Not Started

Multi-Stroke Support

Backstroke

Butterfly

Breaststroke

Phase 5

Not Started

Scientific Validation Expansion

Reference Dataset

Elite Swimmer Benchmarks

International Standards

Phase 6

Not Started

AI Coach

Natural Language Feedback

Training Suggestions

Question Answering

Phase 7

Not Started

Cloud Platform

Authentication

Accounts

Teams

Synchronization

Phase 8

Not Started

Commercial SaaS

Subscriptions

Payments

Organization Management

Phase 9

Not Started

Mobile Applications

Android

iOS

Offline Sync

Engineering Philosophy

Before writing any new code, the engineering team must ask four questions:

Is this scientifically correct? Is it modular? Is it production-ready? Will this still work after five years?

If any answer is *No*, redesign before implementation.

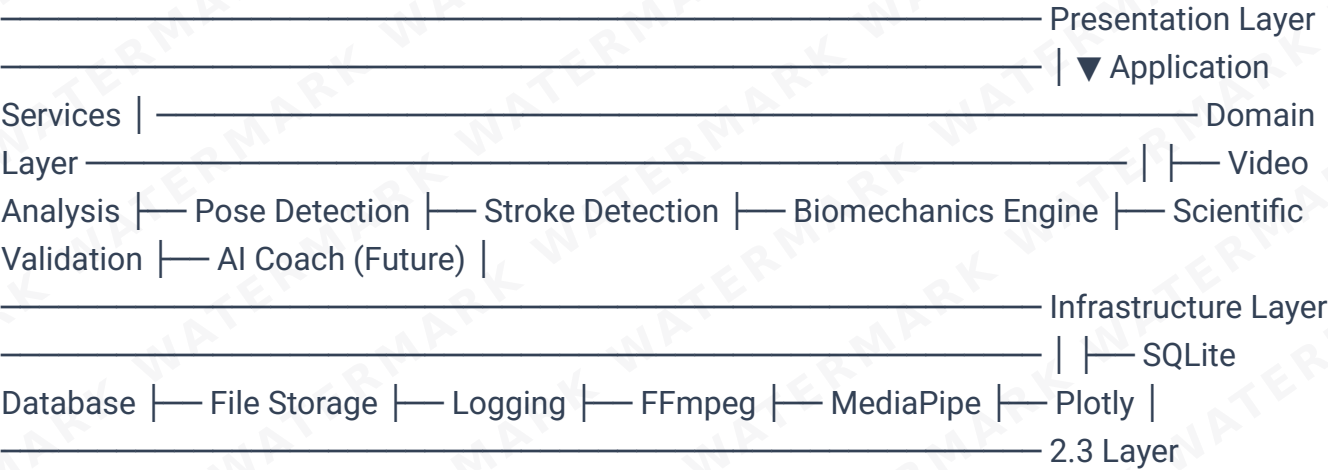
Chapter 2 ### Complete System Architecture 2.1 Introduction

SwimAnalyzer AI is designed using a layered modular architecture inspired by Clean Architecture, Domain-Driven Design (**DDD**), and **SOLID** principles.

The primary goal is to isolate responsibilities so that each subsystem can evolve independently without introducing regressions into other components.

This design also enables future migration from a desktop Streamlit application to a cloud-native SaaS platform with minimal architectural changes.

2.2 High-Level Architecture **USER** | ▼ Streamlit Web Interface |



Responsibilities

Presentation Layer

Responsible only for:

User Interface

Rendering Navigation Charts ### Progress Bars Notifications

Must **NEVER** contain:

SQL queries OpenCV logic MediaPipe logic Mathematical calculations Business rules

The UI should only call service methods.

Application Layer

Coordinates application workflows.

Examples:

Analyze Video

↓

Validate Input



Run Analysis Pipeline



Save Results



Return Dashboard

No UI code exists here.

Domain Layer

The heart of the system.

Contains:

Stroke Analysis

Biomechanics ### Video Quality ### Consistency Validation ### Athlete Models ###
Scientific Rules

Everything here must remain independent from Streamlit.

Infrastructure Layer

Responsible for communication with external systems.

Examples:

SQLite

FFmpeg

MediaPipe

Filesystem

Logging

Future:

PostgreSQL

Redis

Azure Blob

AWS S3

2.4 Data Flow

The complete analysis pipeline follows this sequence:

Upload Video

↓

Validation

↓

Video Preparation

↓

Frame Extraction

↓

MediaPipe Pose Detection

↓

Stroke Analyzer

↓

Biomechanics Calculator

↓

Video Quality Assessment

↓

Scientific Consistency Validation

↓

Performance Score



Annotated Video Export



JSON Report



SQLite Database



Dashboard

Each stage produces structured outputs consumed by the next stage.

2.5 Folder Architecture SwimAnalyzer_AI/

app/ streamlit_app.py

analysis/

services/

core/

config/

database/

repositories/

models/

utils/

tests/

docs/

data/

logs/

exports/

app/static/ 2.6 Core Modules ### Video Processor

Responsibilities

Read videos

Validate format

Frame extraction

Transcoding

Browser compatibility

Generate annotated video

Future

GPU decoding

Batch processing

Pose Detector

Responsibilities

MediaPipe initialization

Landmark extraction

Visibility confidence

Tracking stability

Future

YOLO integration

OpenPose

MoveNet

Stroke Analyzer

Responsibilities

Phase detection

Cycle counting

Stroke timing

Future

HMM smoothing

Deep Learning classifier

Biomechanics Engine

Responsible for:

Joint Angles

Body Roll

Stroke Length

Stroke Rate

Kick Frequency

Symmetry

Fault Detection

Future

3D biomechanics

Force estimation

Hydrodynamic efficiency

Consistency Validator

Purpose

Protect scientific integrity.

Detect contradictions.

Example

High score

Critical video quality



Impossible



Reduce confidence

Never silently ignore contradictions.

Athlete Management

Stores

Athlete Profile

Age

Gender

Height

Weight

Swimming Style

History

Future

Teams

Academies

Competitions

Medical history

Reporting Engine

Outputs

Annotated video

JSON report

Charts

Performance metrics

Coach summary

Future

PDF reports

Email delivery

Cloud reports

2.7 Database Architecture

Current

SQLite

Future

PostgreSQL

Schema

Athletes



Analysis Sessions



Metrics



Reports



Videos

Relationship

Athlete

↓

Many

↓

Analysis Sessions

↓

One

↓

Report

↓

Many

↓

Metrics 2.8 AI Architecture (Future) ### Analysis Results

↓

Knowledge Layer

↓

LLM Prompt Builder

↓

Large Language Model

↓

AI Coach

↓

Recommendations

↓

Training Plan

The AI never accesses raw video directly.

It receives structured biomechanical metrics only.

This guarantees explainability.

2.9 Design Rules

Every module must satisfy:

Single Responsibility

Dependency Injection

Loose Coupling

High Cohesion

Open/Closed Principle

Interface Segregation

Dependency Inversion

2.10 Forbidden Practices

Never:

- ✗ Put **SQL** inside Streamlit pages
- ✗ Use global variables
- ✗ Duplicate business logic
- ✗ Hardcode thresholds
- ✗ Mix UI with calculations
- ✗ Access database directly from UI
- ✗ Modify analysis results after scientific validation

2.11 Technology Stack Layer Technology UI Streamlit Charts Plotly Computer Vision OpenCV
Pose Detection MediaPipe Video Codec FFmpeg Database SQLite Future Database
PostgreSQL AI OpenAI / Gemini / Claude (planned) Testing Pytest Logging Python Logging
Configuration **YAML** Version Control Git 2.12 Architectural Principles for Future Expansion

The architecture is intentionally designed so that adding a new swimming style should require implementing only:

A new Stroke Strategy
A new Biomechanics Strategy
Scientific thresholds
without modifying the rest of the application.

Likewise, replacing SQLite with PostgreSQL, or Streamlit with React + FastAPI, should require changes only in the Infrastructure and Presentation layers while leaving the Domain layer intact.

```
> **Engineering Principle #1** > > **Every metric produced by SwimAnalyzer AI must be scientific;
```

```
---
```

3.1 Introduction

Artificial Intelligence alone is ****not**** sufficient for elite sports analysis.

SwimAnalyzer AI is ****not**** designed to imitate coaches.

Instead, it acts as a ****scientific assistant**** that measures biomechanical variables objectively

The final coaching decision always remains with the human expert.

```
---
```

3.2 Scientific Hierarchy

All calculations in SwimAnalyzer AI must follow the following hierarchy of authority.

Level 1 – International Governing Bodies (Highest Authority)

These organizations define official swimming rules and competition standards.

- World Aquatics (formerly ****FINA****) – Official competition regulations and stroke rules.
- International Olympic Committee (****IOC****) – Olympic technical regulations.

These references determine:

- Legal stroke definitions
- Competition standards
- Stroke rules
- Event specifications

```
---
```

Level 2 – Coaching Organizations

These organizations define coaching methodology and technique development.

Primary references:

- American Swimming Coaches Association (**ASCA**)
- **USA** Swimming
- British Swimming
- Australian Swimming

These references provide:

- Stroke mechanics
- Training principles
- Performance indicators
- Common technical errors

Level 3 – Scientific Literature

Peer-reviewed biomechanical research forms the foundation of all measurable metrics.

Preferred sources:

- Journal of Biomechanics
- Sports Biomechanics
- Journal of Sports Sciences
- International Journal of Sports Physiology and Performance
- Medicine & Science in Sports & Exercise

Only peer-reviewed publications should be used when defining thresholds.

Level 4 – Internal Engineering Rules

Some engineering decisions are not defined by research.

Examples include:

- Confidence calculations
- Missing landmark handling
- Frame interpolation
- Smoothing filters
- Video quality scoring

These rules must always be documented.

3.3 Scientific Validation Philosophy

Every reported value must satisfy three conditions.

Measurable

Can be measured objectively.

Example:

- Shoulder Angle → Yes
- Swimming Style Beauty → No

Repeatable

Running the same video twice should produce nearly identical results.

Target variation:

< 1%

Explainable

The system must explain:

- How the value was calculated.
- Which landmarks were used.
- Which assumptions were made.

3.4 Scientific Confidence

Every analysis produces two independent outputs.

Performance Score

Measures athlete performance.

Example:

87.4 / 100

Scientific Confidence

Measures trust in the analysis.

Possible levels:

- High
- Medium
- Low
- Critical

Performance and confidence must never be confused.

Example:

Excellent swimmer

- Poor camera

↓

Performance = 92

Confidence = Medium

****NOT****

Performance = 55

This distinction preserves fairness.

3.5 Core Biomechanical Metrics

The current Freestyle engine measures:

Stroke Rate

****Definition****

Number of stroke cycles completed per minute.

****Purpose****

Estimate swimming rhythm.

****Unit****

****SPM****

Stroke Length

Definition

Relative forward distance covered per stroke cycle.

Current Implementation

Relative measurement based on calibrated body proportions.

Future

Absolute meters using camera calibration.

Stroke Cycle Duration

Definition

Time required to complete one full stroke cycle.

Unit

Seconds

Body Roll

Definition

Rotation of the swimmer around the longitudinal axis.

Purpose

Evaluate rotational efficiency.

Elbow Angle

Definition

Internal elbow angle during pull phase.

Purpose

Detect dropped elbow.

Shoulder Angle

****Definition****

Relative shoulder position during stroke phases.

****Purpose****

Stroke symmetry.

Hip Rotation

****Purpose****

Evaluate body alignment.

****Future****

Pelvic rotation estimation.

Kick Frequency

****Definition****

Kick rhythm relative to arm cycles.

****Future****

Kick synchronization analysis.

Stroke Symmetry

Measures:

- Left vs Right consistency

Useful for:

- Injury detection
- Technique imbalance

3.6 Video Quality Metrics

Analysis quality depends heavily on video quality.

Current measurements include:

- Pose Visibility
- Motion Blur
- Brightness
- Contrast
- Camera Stability
- Resolution
- Frame Coverage

Each contributes to an overall **Video Quality Assessment (VQA)**.

3.7 Scientific Consistency Validation

Purpose

Prevent impossible conclusions.

Example 1

Excellent biomechanics

- Critical video quality

↓

Confidence decreases.

Example 2

Zero stroke cycles

- Perfect stroke timing

↓

Contradiction detected.

Example 3

Missing landmarks

- Precise joint angles

↓

Estimated values flagged.

3.8 Calibration

Current Calibration

Relative anthropometric scaling.

Reference:

Shoulder width approximation.

Future Improvements

- User height
- Arm span
- Pool lane width
- Reference object calibration
- Multi-camera calibration
- Stereo vision

3.9 Sources of Measurement Error

The following factors reduce scientific confidence:

- Occlusion
- Splashing
- Camera vibration
- Low ****FPS****
- Motion blur
- Poor lighting
- Perspective distortion
- Partial body visibility
- Loose clothing
- Crowded pools

These factors should affect ****confidence**** rather than altering measured performance.

3.10 Scientific Limitations

Current limitations:

- Single-camera analysis
- 2D landmark estimation
- Relative calibration
- No underwater camera support
- No force estimation
- No drag estimation

These limitations must always be disclosed.

3.11 Future Scientific Expansion

Future versions may include:

- 3D Pose Estimation
- Hydrodynamic Modeling
- Velocity Estimation
- Acceleration Curves
- Stroke Efficiency Index
- Propulsion Estimation
- Energy Cost Estimation
- Fatigue Detection
- Injury Risk Prediction

3.12 Research Policy

Every future metric must satisfy the following checklist before implementation:

- Is it supported by peer-reviewed literature?
- Can it be measured reliably from available data?
- Is the algorithm deterministic?
- Can it be validated against real swimmers?
- Can the system explain how it was calculated?
- Does it improve coaching decisions?

If any answer is ****No****, the metric should ****not**** be included in production.

3.13 Scientific Roadmap

Current Phase

- Freestyle analysis only
- 2D analysis

- Relative measurements
- Video Quality Assessment
- Scientific Consistency Validation

Next Phase

- Add Backstroke
- Add Breaststroke
- Add Butterfly
- Create dedicated biomechanical references for each stroke

Advanced Phase

- Build a reference database for swimmers at different levels (Beginner, Intermediate, Elite)
- Calibrate metrics against real-world swimming datasets
- Develop machine learning models trained on real swimming data

Chapter 4

Engineering Standards & Development Rules

> **Engineering Principle #2** > > **The quality of the software architecture determines the long**

4.1 Introduction

This chapter defines the engineering standards that every contributor, AI assistant, and developer

These rules ensure that the project remains:

- Maintainable
- Scalable
- Testable
- Reliable
- Production-ready

No implementation should violate these standards.

4.2 Engineering Philosophy

Every new feature must satisfy the following priorities:

Scientific correctness

Software quality ## Reliability ## Performance ## User experience

Feature quantity is never prioritized over engineering quality.

4.3 Software Design Principles

The project follows the **SOLID** principles.

Single Responsibility Principle (SRP)

Each class should have one responsibility.

Example:

✓ VideoProcessor

✓ PoseDetector

✓ ConsistencyValidator

✗ VideoProcessor that also saves reports and updates the UI.

Open / Closed Principle

Modules should be open for extension but closed for modification.

Example:

Adding Butterfly analysis should require creating a new strategy instead of modifying the FreestyleAnalyzer.

Liskov Substitution Principle

All stroke analyzers should expose the same interface.

Example

```
python StrokeAnalyzer
```

↓

FreestyleAnalyzer

BackstrokeAnalyzer

ButterflyAnalyzer

BreaststrokeAnalyzer ```

Interface Segregation Principle

Large interfaces should be avoided.

Instead of:

```python Analyzer ```

Prefer:

```python PoseAnalyzer

StrokeAnalyzer

VideoAnalyzer

QualityAnalyzer ```

Dependency Inversion Principle

High-level modules should never depend directly on low-level implementations.

Instead of:

```python Streamlit → SQLite ```

Use

```python Streamlit

↓

Repository

↓

SQLite ```

4.4 Clean Architecture Rules

Dependencies must always point inward.

```
``` Presentation
```

↓

```
Application
```

↓

```
Domain
```

↓

```
Infrastructure ```
```

The Domain layer must never depend on:

- Streamlit
- SQLite
- OpenCV
- Ffmpeg
- Plotly

---

## # 4.5 Folder Organization

Approved project structure

```
``` app/
```

```
analysis/
```

```
services/
```

```
repositories/
```

```
database/
```

```
models/
```

```
utils/
```

```
config/
```

```
tests/
```

```
docs/
```

```
logs/
```

```
data/ ```
```

Never create unrelated files in the project root.

```
---
```

4.6 Naming Conventions

Classes

Use PascalCase

```
``` PoseDetector
```

```
VideoProcessor
```

```
AthleteRepository ```
```

```

```

### Functions

Use snake\_case

```
``` calculate_body_roll()
```

```
detect_stroke_cycles()
```

```
estimate_velocity() ```
```

```
---
```

Variables

Use descriptive names.

Good

```
```python stroke_rate
```

```
body_roll
```

```
video_quality_score ```
```

Bad

```
```python x
```

```
temp
```

```
value

data2 ```

---

Constants

Use uppercase.

``` MAX_VIDEO_SIZE

DEFAULT_FPS

MIN_VISIBILITY_SCORE ```

4.7 Documentation Rules

Every public class must contain:

- Purpose
- Inputs
- Outputs
- Exceptions
- Example

Example

```python
class PoseDetector:
    """
    Extracts pose landmarks from swimming videos.

    Inputs:
    video_path

    Outputs:
    Pose landmarks

    Raises:
    RuntimeError
    """
```

4.8 Configuration Rules

Never hardcode:

Thresholds

Paths

URLs

API Keys

Database names

FPS assumptions

Everything belongs inside:

```
config.yaml
```

4.9 Logging Rules

Every major operation should be logged.

Example

```
### Video Uploaded  
  
### Analysis Started  
  
### Pose Detection Completed  
  
### Stroke Detection Completed  
  
### Analysis Saved  
  
### Export Completed
```

Never log:

Passwords

API keys

Sensitive user information

4.10 Exception Handling

Never ignore exceptions.

Bad

```
try:
    ...
except:
    pass
```

Good

```
try:
    ...
except Exception as e:
    logger.exception(e)
```

4.11 Performance Rules

Avoid

Repeated file reads

Repeated MediaPipe initialization

Duplicate frame processing

Unnecessary object allocation

Prefer

Caching

Single-pass processing

Shared resources

Lazy loading

4.12 Database Rules

UI must never execute **SQL** directly.

Correct architecture

↓

Repository

↓

SQLite ~~~

Repositories are responsible for:

Insert

Update

Delete

Search

Transactions

4.13 AI Development Rules

Large Language Models must never:

Modify scientific calculations.

Invent biomechanical metrics.

Change performance scores.

Instead they may:

Explain results.

Generate recommendations.

Summarize reports.

Answer athlete questions.

Generate training advice.

4.14 Code Review Checklist

Before merging any code verify:

- ☐ No duplicated code
- ☐ Tests pass
- ☐ Logging added
- ☐ Documentation updated
- ☐ Configuration externalized
- ☐ No hardcoded paths
- ☐ No unused imports
- ☐ Performance acceptable
- ☐ Scientific correctness preserved
- ☐ UI unchanged unless required

4.15 Git Workflow

Main Branch

Production only

Develop Branch

Integration branch

Feature Branch

One feature per branch

Example

```
``` feature/backstroke-analysis
```

```
feature/sqlite-migration
```

```
feature/ai-coach
```

bugfix/video-export

refactor/pipeline ```

---

# 4.16 Commit Message Convention

Good

``` feat: add athlete repository

fix: resolve video transcoding bug

refactor: split biomechanics calculator

docs: update architecture guide

test: add stroke validator tests ```

Avoid

``` Update

Fix

Changes

Work

Final ```

---

# 4.17 Testing Policy

Every feature requires:

Unit tests

Integration tests

Regression tests

Performance verification

Manual verification

No feature is considered complete without testing.

---

#### # 4.18 Engineering Quality Gates

A feature cannot be merged until:

- ✓ Code review completed
- ✓ Scientific validation completed
- ✓ Tests passed
- ✓ Documentation updated
- ✓ Logging verified
- ✓ Performance verified
- ✓ UI verified

---

#### # 4.19 Definition of Done

A task is complete only when:

- Implementation finished
- Tests passing
- Documentation updated
- Logs verified
- Performance acceptable
- No regressions introduced
- Scientific behavior validated

---

#### # 4.20 Engineering Commandments

The following rules are mandatory:

## Never sacrifice architecture for speed.

## Every metric must be explainable.

## Never duplicate business logic.

## Keep UI independent from analysis.

## Always write tests.

## Optimize only after measuring.

```
Every warning must be meaningful.
```

```
Every exception must be logged.
```

```
Prefer readability over cleverness.
```

```
Build today as if the software will still be running ten years from now.
```

```

```

```
```markdown id=*engineering-playbook-ch5* # Chapter 5 # Development Roadmap & Project Phases
```

```
> **Engineering Principle #3** > > **Build a stable foundation before expanding functionality. Ev
```

```
---
```

```
# 5.1 Purpose
```

This roadmap defines the official engineering direction of SwimAnalyzer AI.

It serves four purposes:

- Guide future development
- Prevent feature creep
- Prioritize engineering work
- Ensure scientific integrity

Every implemented feature must belong to one roadmap phase.

```
---
```

```
# 5.2 Long-Term Vision
```

The final vision is to transform SwimAnalyzer AI into the world's [leading AI-powered swimming an](#)

- Individual swimmers
- Professional coaches
- Swimming academies
- National teams
- Universities
- Sports science laboratories
- Olympic training centers

The system should evolve from a desktop application into a cloud-native SaaS platform without rec

```
---
```

```
# 5.3 Roadmap Overview
```

```
``` Phase 1 ↓
```

### Core Analysis Engine



Phase 2 ↓

### Athlete Management



Phase 3 ↓

### Performance Tracking



Phase 4 ↓

Multi-Stroke Support



Phase 5 ↓

### Scientific Validation Expansion



Phase 6 ↓

AI Coach



Phase 7 ↓

### Cloud Platform



Phase 8 ↓

Commercial SaaS



Phase 9 ↓

### Mobile Applications





Phase 10 ↓

### Research Platform

---

## 5.4 Phase 1 – Core Analysis Engine

---

### Objective

---

Develop a stable scientific pipeline capable of analyzing freestyle swimming videos.

### Completed

- ✓ Video upload
- ✓ Video validation
- ✓ Pose detection
- ✓ Stroke cycle detection
- ✓ Biomechanics engine
- ✓ Video quality assessment
- ✓ Scientific consistency validation
- ✓ Annotated video generation
- ✓ **JSON** reports
- ✓ Logging system
- ✓ Crash-free architecture
- ✓ Browser-compatible H.264 export
- ✓ Single-pass processing pipeline

Status:

**Completed**

---

## 5.5 Phase 2 – Athlete Management

---

### Objective

---

Transform the application from video analysis into athlete-centered analysis.

### Completed

- ✓ Athlete profiles
- ✓ SQLite database
- ✓ Session history
- ✓ Historical metrics
- ✓ Plotly dashboards
- ✓ Analysis persistence

Status:

**Completed**

---

## 5.6 Phase 3 – Performance Tracking

---

### Objective

---

Enable coaches to monitor swimmer progression over time.

### Features

- ✓ Historical charts
- ✓ Athlete dashboard
- ✓ Performance timeline
- ✓ Session comparison
- ✓ Metric trends

Remaining:

- ☐ Performance trend prediction
- ☐ Personal best tracking
- ☐ Seasonal summaries

Status:

**In Progress**

---

## 5.7 Phase 4 — Multi-Stroke Support

---

### Objective

---

Expand beyond freestyle.

Target strokes:

- ☐ Backstroke
- ☐ Breaststroke
- ☐ Butterfly

Each stroke requires:

- Dedicated analyzer
- Stroke phases
- Biomechanics calculator
- Scientific thresholds
- Unit tests
- Validation dataset

No modifications should be required in the UI.

Status:

**Planned**

---

## 5.8 Phase 5 – Scientific Validation Expansion

---

### Objective

---

Strengthen scientific credibility.

Planned features:

- ☐ Elite swimmer reference dataset
- ☐ Age-specific benchmarks
- ☐ Gender-specific normalization
- ☐ Anthropometric calibration
- ☐ Reference ranges
- ☐ Research citations
- ☐ Confidence calibration

Status:

**Planned**

---

## 5.9 Phase 6 – AI Coach

---

### Objective

---

Transform measurements into coaching intelligence.

Capabilities:

- ☐ Explain analysis
- ☐ Generate recommendations
- ☐ Answer athlete questions
- ☐ Identify priorities
- ☐ Build training plans

- ☐ Explain errors
- ☐ Weekly coaching reports

### Important Rule

The AI must never modify scientific measurements.

It may only interpret validated results.

Status:

**Planned**

---

## 5.10 Phase 7 – Cloud Platform

---

### Objective

---

Convert the desktop application into a cloud platform.

Components:

- ☐ User accounts
- ☐ Authentication
- ☐ Cloud database
- ☐ Cloud storage
- ☐ Online dashboards
- ☐ Coach portals
- ☐ Athlete portals
- ☐ **API**
- ☐ Synchronization

Technology candidates:

FastAPI

PostgreSQL

Redis

Docker

Nginx

Status:

**Planned**

---

## 5.11 Phase 8 – Commercial SaaS

---

### Objective

---

Launch a production-ready commercial platform.

Capabilities:

- ☐ Subscription system
- ☐ Stripe integration
- ☐ Team management
- ☐ Organization management
- ☐ Coach permissions
- ☐ Athlete permissions
- ☐ Billing
- ☐ Usage analytics
- ☐ Enterprise licensing

Status:

**Planned**

---

## 5.12 Phase 9 – Mobile Applications

---

Platforms

Android

iOS

Capabilities

☐ Video upload

☐ Training history

☐ Notifications

☐ AI Coach chat

☐ Offline synchronization

☐ Cloud synchronization

Future technologies

Flutter

**React Native**

**Native Swift**

**Native Kotlin**

Status:

**Planned**

---

## 5.13 Phase 10 – Research Platform

### Objective

---

Become a scientific platform for swimming biomechanics research.

Capabilities

☐ Research datasets

- ☐ Statistical analysis
- ☐ **CSV** export
- ☐ Multi-athlete comparison
- ☐ Machine learning datasets
- ☐ Research **API**
- ☐ University partnerships
- ☐ Benchmark publications

Status:

**Future Vision**

---

## 5.14 Current Project Status

---

### Completed

---

- Core Analysis Engine
- Database Integration
- Athlete Management
- Historical Tracking
- Plotly Visualization
- Production Stability
- Logging
- Single-Pass Pipeline

### Overall Progress

Approximately **40–45%** of the long-term roadmap.

The remaining work focuses primarily on expanding scientific capabilities, AI integration, cloud deployment, and commercialization.

---

## 5.15 Phase Dependencies

---



### Core Engine



### Athlete Management



### Performance Tracking



Multi-Stroke Support



### Scientific Validation



AI Coach



### Cloud Platform



Commercial SaaS



### Mobile Apps

No later phase should begin before the dependencies of earlier phases are stable.

## 5.16 Development Priorities

Priority 1

Scientific correctness

Priority 2

System stability

Priority 3

Performance optimization

---

Priority 4

User experience

---

Priority 5

New features

New functionality should never compromise scientific validity or system reliability.

---

## 5.17 Success Metrics

---

Technical

- ✓ Crash-free operation
- ✓ Stable processing
- ✓ High test coverage
- ✓ Modular architecture

Scientific

- ✓ Explainable metrics
- ✓ Peer-reviewed references
- ✓ Confidence validation

Business

- ✓ Multi-user platform
- ✓ Subscription-ready
- ✓ Cloud-ready
- ✓ Enterprise-ready

---

## 5.18 Engineering Rule

---

Before implementing any roadmap item, verify:

- Does it align with the long-term vision?
- Does it preserve modular architecture?
- Does it improve scientific value?
- Can it be tested objectively?
- Will it still be maintainable in five years?

If any answer is **No**, redesign before implementation.

---

````markdown id=*engineering-playbook-ch6* # Chapter 6 # Scientific Foundation & Reference Framework

Scientific Principle #1 > > Every metric produced by SwimAnalyzer AI must be scientifically explainable, reproducible, and traceable to accepted biomechanics research.

6.1 Purpose

Unlike ordinary AI applications, SwimAnalyzer AI is classified as a **scientific decision-support system**.

This means that every score, warning, recommendation, and biomechanical metric must be supported by established sports science—not by intuition or arbitrary AI outputs.

This chapter defines the scientific framework that governs the entire project.

6.2 Scientific Philosophy

The application **does not invent new biomechanics**.

Instead, it measures, estimates, validates, and explains swimming technique using accepted scientific knowledge.

Every calculation must satisfy three requirements:

- Scientifically explainable
- Computationally reproducible
- Biomechanically meaningful

6.3 Scientific Reference Pyramid



The AI layer is always the **last** layer—not the source of truth.

6.4 Primary Scientific Domains

The project combines knowledge from several disciplines.

Swimming Biomechanics

Focuses on:

- Stroke mechanics
 - Joint kinematics
 - Propulsion
 - Hydrodynamics
 - Efficiency
-

Sports Science

Includes:

- Performance analysis
 - Motor learning
 - Athlete development
 - Training adaptation
 - Fatigue assessment
-

Computer Vision

Responsible for:

- Pose estimation
 - Landmark tracking
 - Motion extraction
 - Frame synchronization
-

Artificial Intelligence

Responsible only for:

- Pattern interpretation
- Report explanation
- Coaching recommendations
- Natural language interaction

AI is **not responsible** for calculating biomechanics.

6.5 Official Scientific Sources

The project should prioritize evidence from:

FINA / World Aquatics

Official swimming regulations

Technique standards

Competition rules

International Olympic Committee

Athlete development

Competition standards

Training recommendations

Peer-Reviewed Journals

Examples include:

Sports Biomechanics

Journal of Biomechanics

Journal of Sports Sciences

Medicine & Science in Sports & Exercise

Frontiers in Sports and Active Living

IEEE (Computer Vision)

Nature (when applicable)

University Research

Research from internationally recognized institutions should be preferred over blogs or commercial websites.

6.6 Scientific Confidence Levels

Every reported metric should include an internal confidence level.

High Confidence

Excellent pose visibility

High landmark reliability

Complete stroke cycles

Stable tracking

Medium Confidence

Minor occlusions

Small tracking instability

Partial stroke visibility

Moderate video quality

Low Confidence

Poor lighting

Heavy occlusions

Incomplete cycles

Low visibility

Camera instability

Critical

Analysis should not produce scientific conclusions.

Only diagnostic feedback should be shown.

6.7 Evidence Chain

Every metric follows the same scientific chain.



Pose Landmarks



Biomechanical Measurement



Validation



Consistency Check



Scientific Confidence



Final Report

Skipping any step is prohibited.

6.8 Scientific Validation Rules

Every metric must answer:

What was measured?

Example:

Elbow angle

How was it measured?

Pose landmarks

Which algorithm calculated it?

Biomechanics calculator

How reliable is it?

Consistency validator

Is the measurement scientifically meaningful?

Compared against accepted biomechanics.

6.9 Measurement Categories

Direct Measurements

Obtained directly from pose landmarks.

Examples

Joint angle

Frame count

Stroke duration

Body orientation

Estimated Measurements

Require mathematical estimation.

Examples

Stroke length

Swimming velocity

Body roll

Kick frequency

Relative Measurements

Depend on camera calibration.

Examples

Distance

Velocity

Reach

Trajectory

These should always be marked as estimated unless anthropometric calibration is available.

6.10 Anthropometric Calibration

Current status

Relative calibration only.

Future roadmap

User-provided:

Height

Shoulder width

Arm span

Leg length

These values will improve:

Stroke length

Velocity

Distance estimation

Efficiency calculations

6.11 Video Quality Constraints

Scientific analysis is only meaningful when the video satisfies minimum requirements.

Factors include:

Lighting

Resolution

Frame rate

Camera stability

Pose visibility

Occlusion level

If quality is insufficient, confidence—not technique score—should be reduced.

6.12 Scientific Integrity Rules

The system must never:

Invent landmarks

Guess stroke phases without evidence

Hide uncertainty

Modify measurements to satisfy expectations

Generate unsupported coaching advice

When uncertainty exists, it must be explicitly reported.

6.13 AI Usage Policy

AI is allowed to:

Explain metrics

Summarize reports

Suggest training priorities

Answer athlete questions

Generate natural language

AI is **not allowed** to:

Modify scientific scores

Invent measurements

Override biomechanics

Change confidence levels

Create unsupported medical advice

6.14 Scientific Expansion Roadmap

Future reference datasets should include:

Elite swimmers

Junior swimmers

Masters swimmers

Male athletes

Female athletes

Open-water swimmers

Para swimmers

This enables age- and population-specific benchmarking.

6.15 Future Benchmark System

Instead of displaying only raw metrics, the platform should compare athletes against reference populations.

Example:

```
### Current Athlete
```

↓

Compared to Elite Dataset

↓

72nd Percentile ...

This provides meaningful context for coaches.

```
# 6.16 Research Traceability
```

Every scientific module should maintain a mapping between:

Metric

↓

Algorithm

↓

Reference

↓

Version

↓

```
### Validation Tests
```

This ensures reproducibility and simplifies future peer review.

```
# 6.17 Scientific Quality Gates
```

Before introducing any new biomechanical metric, `verify`:

✓ Is it supported by scientific literature?

✓ Can it be measured objectively?

✓ Is the algorithm reproducible?

✓ Is uncertainty quantified?

✓ Can the result be explained to a coach?

If any answer is **No**, the metric should not be included.

6.18 Future Scientific Vision

The long-term goal is to evolve SwimAnalyzer AI from a measurement tool into a scientifically valid

- Athlete development
- Coach education
- Research collaboration
- Performance benchmarking
- Large-scale swimming analytics

The platform should eventually be suitable for use in academic research and elite training environments.

End of Chapter 6

This chapter establishes the scientific foundation of SwimAnalyzer AI.

All future algorithms, AI features, and performance metrics must remain consistent with these principles.

> **Engineering Principle #4** > > **A scalable system is built from independent modules that communicate through well-defined interfaces.**

7.1 Purpose

This chapter defines the official architecture of SwimAnalyzer AI.

It serves as the engineering blueprint that governs how every component communicates, how data flows, and how the system evolves.

Every new module must follow this architecture.

7.2 Architectural Philosophy

SwimAnalyzer AI follows a layered architecture based on Separation of Concerns (SoC).

Each layer has a single responsibility.

...
Presentation Layer

↓

Application Layer

↓

Analysis Layer

↓

Domain Layer

↓

Infrastructure Layer

Each layer communicates only with the layer directly beneath it.

7.3 High-Level Architecture

↓

Streamlit UI

↓

Application Services

↓

Analysis Pipeline

↓

Scientific Validation

↓

Persistence Layer

↓

Reports & Visualization ---

No UI component should perform scientific calculations.

No analysis component should know how the UI is rendered.

7.4 Complete System Diagram

User

|

▼

Streamlit **Interface**

|

▼

Application Services

|

├──

▼

|

▼

|

▼

Video Athlete Configuration Upload Management Loader | | |

|

|

▼

Analysis Service

|

|

▼

▼▼ Video Quality Pose Calibration Assessment Detection Engine | |

|

|

▼

Stroke Analyzer Factory

|

|

▼▼▼

Freestyle Backstroke Butterfly Analyzer Analyzer Analyzer | ▼ Biomechanics Calculator | ▼

Consistency Validator | ▼ Performance Scoring

|

|

▼▼▼

JSON

Video Export Database Report Renderer Storage | ▼ Plotly Dashboards | ▼ End User

7.5 Layer Responsibilities

Presentation Layer

Responsible for:

- User Interface
- Navigation
- Upload controls
- Charts

- Video playback
- Reports

Must never:

- Calculate biomechanics
- Read database directly
- Execute MediaPipe

Application Layer

Responsible for:

- Workflow orchestration
- Session management
- Progress updates
- Error handling

Examples

AnalysisService

AthleteService

ExportService

Analysis Layer

Responsible for:

- Pose estimation
- Stroke detection
- Joint angles
- Performance metrics
- Scientific calculations

This is the scientific core of the application.

```
## Domain Layer
```

```
Contains:
```

```
Entities
```

```
Models
```

```
Business rules
```

```
Validation logic
```

```
Examples
```

```
Athlete
```

```
AnalysisSession
```

```
StrokeMetrics
```

```
ConsistencyResult
```

```
---
```

```
## Infrastructure Layer
```

```
Responsible for:
```

- SQLite
- File storage
- **JSON** export
- Logging
- Configuration
- FFmpeg
- Media files

```
No scientific calculations belong here.
```

```
---
```

```
# 7.6 Current Project Structure
```

```
... app/

analysis/

services/

core/

database/

repositories/

models/

utils/

config/

tests/

docs/

data/

logs/

static/ ...
```

Each directory has a single responsibility.

```
---
```

```
# 7.7 Data Flow
```

```
... ### Video Upload
```

```
↓
```

```
Validation
```

```
↓
```

```
### Frame Extraction
```

↓

Pose Detection

↓

Stroke Analysis

↓

Biomechanics

↓

Scientific Validation

↓

Score Calculation

↓

Database Storage

↓

Charts

↓

Report Export

Every analysis follows this exact order.

7.8 Analysis Pipeline

↓

Video Quality Assessment

↓

Pose Landmarks

↓

Stroke Classification

↓

Biomechanical Metrics

↓

Consistency Validation

↓

Performance Score

↓

Export ```

This pipeline is deterministic.

The same input video should always produce the same result.

7.9 Database Architecture

Current Database

SQLite

Future

PostgreSQL

Domain entities:

``` Athlete

↓

### Analysis Session

↓

Reports



Videos



Metrics ...

Relationships

... Athlete

1



Many



### Analysis Sessions



One



Report ...

---

# 7.10 Strategy Pattern

Each swimming style has its own analyzer.

... ### Stroke Analyzer



### Freestyle Analyzer



### Backstroke Analyzer



### Breaststroke Analyzer

↓

```
Butterfly Analyzer
```

The application never checks stroke type using large if statements.

Instead, it delegates to the appropriate strategy.

---

## 7.11 Repository Pattern

---

Repositories isolate business logic from database implementation.

↓

```
Athlete Repository
```

↓

```
SQLite
```

Future

↓

```
Athlete Repository
```

↓

```
PostgreSQL
```

The UI never changes.

Only repository implementation changes.

---

## 7.12 Configuration System

---

All configurable values belong in:

config.yaml

Examples

Thresholds

Confidence values

Video quality limits

Calibration settings

AI settings

Database settings

Magic numbers should never appear inside algorithms.

---

## 7.13 Logging Architecture

---

↓

Logger

↓

### File Handler

↓

logs/app.log ````

Every important action should be logged.

Examples

Application startup

Video upload

Analysis start

Analysis finish

Errors



Warnings

Database events

---

# 7.14 AI Integration Architecture

Future AI layer

``` ### Scientific Report

↓

AI Coach

↓

Natural Language

↓

Athlete ```

The AI never interacts directly with MediaPipe or biomechanics calculations.

It only receives validated results.

7.15 Scalability

Current

Desktop

↓

Future

Cloud

↓

Enterprise

↓

Global SaaS

The architecture should require minimal changes during this evolution.

7.16 Extension Rules

Before adding a new feature, **determine**:

Which layer owns it?

Does it require a new service?

Does it require a repository?

Does it require scientific validation?

Can it be unit tested?

No feature should bypass architectural boundaries.

7.17 Dependency Rules

Allowed

``` UI

↓

Services

↓

Analysis

↓

Repositories

↓

Database ```

Forbidden

``` UI

↓

Database ```

Forbidden

``` Analysis

↓

Streamlit ```

Forbidden

``` Database

↓

Streamlit ```

These rules preserve maintainability.

7.18 Long-Term Architecture Vision

As the platform grows, the same architecture should support:

- Desktop application
- Cloud platform
- Mobile applications
- **REST** **API**
- Research platform
- Enterprise deployment

without rewriting the scientific core.

The analysis engine should become a reusable component independent of the user interface.

End of Chapter 7

This architecture is the structural foundation of SwimAnalyzer AI.

Every future module, service, database component, and AI feature must comply with these architectural

> **Engineering Principle #5** > > **Data is one of the most valuable assets of the platform. The

8.1 Purpose

This chapter defines the official data architecture of SwimAnalyzer AI.

It specifies:

- Data ownership
- Database design
- Relationships
- Storage strategy
- Versioning
- Future migration path

Every feature added to the project must integrate into this architecture.

8.2 Data Philosophy

The platform does ****not**** analyze videos.

It analyzes ****athletes****.

Videos are only evidence.

Reports are only observations.

The athlete is the central entity.

``` Athlete

↓

### Analysis Sessions

↓

Reports

↓

Metrics

↓

Progress ```

Everything revolves around the athlete.

---

## # 8.3 Current Database

Current engine

```
``` SQLite ```
```

Reasons

- Zero configuration
- Lightweight
- **ACID** compliant
- Cross-platform
- Reliable
- Perfect for desktop deployments

SQLite remains the official database until cloud deployment.

```
---
```

```
# 8.4 Future Database
```

Future production database

```
``` PostgreSQL ```
```

Reasons

- Multi-user
- Concurrent writes
- Replication
- Backup support
- Scalability
- Enterprise ready

The Repository Pattern ensures that the migration requires minimal code changes.

```

```

```
8.5 Domain Model
```

The platform revolves around five primary entities.

```
``` Athlete
```

```
↓
```

```
### Analysis Session
```

```
↓
```

```
### Analysis Report
```

```
↓
```

Video



Performance History

Future entities

Coach

Team

Training Plan

Competition

AI Recommendation

8.6 Athlete Entity

Represents one swimmer.

Core fields

first_name

last_name

gender

date_of_birth

height

weight

arm_span

shoulder_width

created_at ...

Future additions

Dominant arm

Competition level

Club

Coach

Medical notes

8.7 Analysis Session

Represents one processed video.

Fields

``` session\_id

athlete\_id

date

stroke\_type

video\_path

processed\_video\_path

report\_path

analysis\_version

duration

overall\_score

confidence ```

Each uploaded video creates exactly one session.

---

# 8.8 Analysis Report

Contains scientific measurements.

Examples

``` ### Stroke Rate

Stroke Length

Body Roll

Kick Frequency

Symmetry

Efficiency

Joint Angles

Consistency

Warnings

Performance Score

Reports remain immutable after creation.

8.9 Video Entity

Every session references:

Original video

Processed video

Thumbnail (future)

Metadata

Future cloud storage

Video **URL**

Cloud object ID

Checksum

8.10 Performance History

Instead of recalculating historical values, every session becomes part of the athlete timeline.

↓

Session 1

↓

Session 2

↓

Session 3

↓

Session 4

↓

Trend Analysis

Historical reports are never overwritten.

8.11 Entity Relationships

1

↓

Many

↓

Analysis Sessions

↓

One

↓

Analysis Report

↓

One

↓

Processed Video

Future

↓

Many Athletes

↓

Many Sessions

8.12 Repository Pattern

The UI never accesses SQLite directly.

↓

Repository

↓

SQLite

Repositories own:

Insert

Update

Delete

Queries

Transactions

Business logic never contains **SQL**.

8.13 Current Storage Layout

```
`` data/  
  swim_analyzer.db  
  
  input_videos/  
  
  output_videos/  
  
  reports/  
  
  exports/
```

Future

```
athletes/  
  athlete_id/  
    sessions/  
      videos/  
        reports/  
          thumbnails/
```

This improves organization for cloud migration.

8.14 Data Lifecycle

↓

Validate

↓

Analyze

↓

```
### Generate Report
```

↓

```
### Store Database
```

↓

```
### Generate Charts
```

↓

```
### Historical Tracking
```

Nothing is discarded unless explicitly deleted.

8.15 Data Versioning

Every report stores

```
application_version
```

```
model_version
```

```
configuration_version` ``
```

This allows future comparisons between algorithm versions.

```
---
```

```
# 8.16 Scientific Traceability
```

Each report should contain

```
```` Metric
```

↓

```
Algorithm
```

↓

```
Confidence
```

↓

Reference

↓

Timestamp ---

Future research depends on reproducible data.

---

# 8.17 Database Constraints

Mandatory rules

Every athlete has a unique ID.

Every session belongs to exactly one athlete.

Reports cannot exist without a session.

Processed videos cannot exist without a report.

Foreign keys should always be enforced.

---

# 8.18 Backup Strategy

Desktop

Automatic database backup

Weekly export

Manual export

Cloud

Daily backups

Incremental backups

Point-in-time recovery

---

# 8.19 Data Integrity Rules

Never delete reports automatically.

Never overwrite scientific measurements.

Never modify historical sessions.

Corrections should generate a new version.

Historical evidence must remain intact.

---

# 8.20 Future Cloud Storage

Current

``` SQLite

- ### Local Files

Future

- ****AWS**** S3

or

Azure Blob

or

Google Cloud Storage

Videos should eventually be stored independently from the relational database.

8.21 Data Security

Sensitive data includes

Athlete identity

Birth date

Body measurements

Performance history

Future cloud deployments must support

Encryption at rest

Encrypted backups

Role-based permissions

Secure authentication

Audit logs

8.22 Scalability Strategy

Current

Thousands of sessions

Future

Millions of sessions

Architecture decisions today must support future growth without redesigning the database.

8.23 Engineering Rules

Every new entity must answer

Who owns it?

Who creates it?

Who updates it?

Who deletes it?

How is it related?

Can it be versioned?

Can it be tested?

Only then should implementation begin.

8.24 Long-Term Data Vision

Eventually, SwimAnalyzer AI should become one of the largest structured swimming biomechanics databases in the world.

The accumulated historical data will enable:

- Athlete progression analysis
- Team analytics
- AI model training
- Scientific research
- Injury prevention studies
- International benchmarking
- Coaching intelligence

The database is not merely storage—it is the foundation of the platform's long-term intelligence.

End of Chapter 8

This chapter defines the official data architecture of SwimAnalyzer AI.

Every future feature, AI capability, cloud service, and research module must integrate into this data model to preserve consistency, scalability, and scientific traceability.

`id=engineering-playbook-ch9 # Chapter 9 # Artificial Intelligence Strategy`

AI Principle #1 > > Artificial Intelligence does not replace scientific analysis—it amplifies its value by transforming validated measurements into understandable, actionable knowledge.

9.1 Purpose

This chapter defines the long-term Artificial Intelligence strategy for SwimAnalyzer AI.

It specifies:

- Where AI is allowed
- Where AI is forbidden
- Which models should be used

- How AI integrates with the scientific engine
- The future roadmap toward an AI-powered coaching platform

9.2 AI Philosophy

The platform has two independent engines:

```
- ### Artificial Intelligence Engine
```

They serve different purposes.

The scientific engine produces facts.

The AI engine explains those facts.

9.3 Core Principle

The scientific pipeline always runs first.

```
↓  
### Scientific Analysis  
↓  
### Validated Metrics  
↓  
### Consistency Validation  
↓  
### Performance Score  
↓  
### Artificial Intelligence
```

AI never receives raw video.

AI never performs biomechanics.

AI only receives validated scientific results.

9.4 Responsibilities of AI

The AI engine is responsible for:

- Explaining metrics
 - Explaining errors
 - Generating coaching advice
 - Answering athlete questions
 - Summarizing reports
 - Producing training recommendations
 - Explaining long-term progress
 - Generating weekly and monthly summaries
-

9.5 Responsibilities Forbidden to AI

AI must never:

- ✗ Calculate joint angles
- ✗ Detect pose landmarks
- ✗ Modify performance scores
- ✗ Change confidence values
- ✗ Invent scientific measurements
- ✗ Replace validation logic
- ✗ Ignore uncertainty
- ✗ Generate medical diagnoses

Scientific integrity always takes precedence over AI creativity.

9.6 AI Architecture

↓

Prompt Builder

↓

LLM

↓

Coach Response

↓

User ```

The AI layer is completely isolated from the scientific engine.

9.7 AI Input

The AI should receive structured data only.

Example

```json

```
{
 overall_score: 87.5,
 stroke_rate: 34.2,
 body_roll: 43.1,
 symmetry: 91,
 confidence: *High*,
 warnings: [
 Early hand exit,
 Low elbow during pull
]
}
```

The AI should never parse raw MediaPipe landmarks.

---

## 9.8 AI Output

---

Expected outputs include:

### **Athlete Summary**

Simple explanation of the session.

---

### **Coach Summary**

Detailed biomechanical interpretation.

---

### **Improvement Priorities**

Top three areas requiring attention.

---

### **Strengths**

Positive aspects of the swimmer.

---

### **Weaknesses**

Technique issues supported by measurements.

---

### **Suggested Training Focus**

Training recommendations based on validated results.

---

## **9.9 Prompt Engineering**

---

Every prompt should contain four sections:

↓

### Scientific Context

↓

### Validated Metrics

↓

### Response Instructions

This structure improves consistency and reduces hallucinations.

---

## 9.10 AI Personas

---

The platform should support multiple personas.

### **Athlete Coach**

Simple language.

Motivational tone.

Minimal technical terminology.

---

### **Professional Coach**

Advanced biomechanics.

Training terminology.

Detailed explanations.

---

### **Sports Scientist**

Scientific interpretation.

Confidence discussion.

Measurement methodology.

Research-oriented language.

---

### **Parent Mode**

Simple explanations suitable for young athletes and parents.

---

## 9.11 AI Memory

---

Future versions may maintain contextual memory for each athlete.

Examples

Previous sessions

Recurring weaknesses

Improvement history

Training recommendations

Coach notes

The AI should recognize long-term trends without modifying historical data.

---

## 9.12 AI Training Plan Generator

---

Future capability

Input

Validated reports

↓

Progress history

↓

Training priorities

↓

Generated weekly plan

The generated plan should always indicate that it is advisory.

---

## 9.13 AI Chat Assistant

---

Future users should be able to ask questions such as:

*Why did my score decrease?*

*What should I improve first?*

*How can I increase stroke length?*

*Explain body roll.*

*What changed compared to last month?*

The AI should answer using only validated report data.

---

## 9.14 AI Model Strategy

---

The platform should remain model-independent.

Supported providers may include:

OpenAI

Anthropic

### **Google Gemini**

Open-source Llama models

Future local models

Changing providers should require minimal code changes.

---

## 9.15 AI Service Layer

---

Recommended architecture



### Provider Adapter

↓

OpenAI

Claude

Gemini

Llama ...

The application communicates only with the interface.

---

# 9.16 Hallucination Prevention

The AI must never answer beyond the available evidence.

If information is unavailable, it should respond with:

> "This information cannot be determined from the available scientific analysis."

Unknown information should never be invented.

---

# 9.17 Scientific References in AI

Whenever possible, the AI should reference accepted biomechanics concepts rather than unsupported

Example

Instead of

> \*Your body roll is bad.\*

Use

> "The measured body roll is below the typical range observed in efficient freestyle swimmers, wh

---

# 9.18 AI Safety Rules

The AI must refuse to:

Provide medical diagnoses.

Predict injuries with certainty.



Recommend medication.

Replace professional coaching.

Guarantee competition results.

Interpret information not supported by the report.

---

#### # 9.19 Long-Term AI Roadmap

Future capabilities include:

- ☐ Conversational AI Coach
- ☐ Automatic training plans
- ☐ Season progress analysis
- ☐ Competition preparation
- ☐ Goal tracking
- ☐ Performance forecasting
- ☐ Team-level insights
- ☐ Coach assistant
- ☐ Research assistant
- ☐ Voice interaction

---

#### # 9.20 Engineering Rules

Before adding any AI feature, [verify](#):

- ✓ Does it use validated scientific outputs?
- ✓ Does it preserve explainability?
- ✓ Can every statement be traced to evidence?
- ✓ Does it avoid hallucination?
- ✓ Does it preserve scientific integrity?

If any answer is **\*\*No\*\***, redesign the feature before implementation.

---

#### # 9.21 Long-Term Vision

The ultimate goal is not to create another chatbot.

The goal is to create the world's first AI Swimming Performance Assistant built upon a scientific

The AI should become a trusted partner for:

- Swimmers
- Coaches
- Sports scientists
- Researchers
- National federations

while always remaining subordinate to the scientific evidence generated by the core analysis eng:

---

#### ## End of Chapter 9

This chapter defines the official AI strategy of SwimAnalyzer AI.

All future AI modules, prompts, models, and conversational interfaces must comply with these pri

> **\*\*Engineering Principle #6\*\*** > > **\*\*A** feature is not complete when it works—it is complete when

---

#### # 10.1 Purpose

This chapter defines the official testing strategy for SwimAnalyzer AI.

Every algorithm, feature, UI component, AI module, and database operation must be verifiable thro

Testing is mandatory—not optional.

---

#### # 10.2 Quality Philosophy

SwimAnalyzer AI is a scientific platform.

Therefore, every release must satisfy three conditions:

``` Correct

↓

Reliable



Repeatable ---

Passing visual inspection alone is never sufficient.

10.3 Testing Pyramid

Manual Validation



End-to-End Tests



Integration Tests



Unit Tests

Most tests should exist at the Unit level.

10.4 Testing Categories

The project uses five major testing categories.

Unit Tests

Verify individual algorithms.

Examples

Joint angle calculation

Stroke detection

Consistency validator

Video utilities

Database repository

Integration Tests

Verify modules working together.

Examples

Pose Detection

↓

Biomechanics

↓

Consistency Validation

↓

Final Score

End-to-End Tests

Validate the complete workflow.

Example

Upload Video

↓

Analyze

↓

Store Database

↓

Generate Charts

↓

Export Report

↓

Render Dashboard

Regression Tests

Guarantee existing behavior remains unchanged after refactoring.

Used whenever:

Performance optimization

Bug fixes

Architecture changes

Manual Scientific Validation

Human verification performed by developers, coaches, or researchers.

10.5 Current Test Coverage

Current automated tests include:

✓ Consistency Validator

✓ Validation Math

✓ Video Export

✓ Freestyle Pipeline

✓ Video Quality Assessment

Future coverage should include:

☐ Athlete Repository

☐ Streamlit UI

☐ AI Coach

☐ Authentication

☐ Cloud ****API****

10.6 Unit Testing Rules

Every algorithm must satisfy:

Deterministic output

Known expected values

Independent execution

No external dependencies

Fast execution

Unit tests should not require:

Internet

****GPU****

Database server

Streamlit

10.7 Scientific Validation Tests

Unlike traditional software, scientific correctness must also be tested.

Examples

High-quality swimmer

↓

High score

↓

High confidence

Poor technique

↓

Lower score

↓

Correct warning generation

Poor video quality

↓

Confidence decreases

↓

Technique score remains fair

These scenarios ensure scientific integrity.

10.8 Regression Protection

Whenever an algorithm changes:

Run all existing tests.

Compare previous outputs.

Confirm **no** unintended differences.

Scientific algorithms should never change silently.

10.9 Performance Testing

Each release should measure:

Analysis duration

Memory usage

****CPU**** usage

Frame processing speed

Video export time

Database response

Future ****KPI**** example

``` **\*\*210\*\***-frame video

↓

Less than **15** seconds ```

---

#### # 10.10 Stress Testing



The platform should be tested with:

Large videos

Multiple athletes

Long sessions

High-resolution recordings

Repeated analyses

Future cloud deployment should include concurrent users.

---

# 10.11 Database Testing

Verify:

Insert athlete

Update athlete

Delete athlete

Insert session

Retrieve history

Foreign keys

Repository mapping

Transactions

---

# 10.12 UI Testing

Future Streamlit AppTest coverage should verify:

Navigation

Athlete creation

Video upload

Analysis execution

Charts

History

Downloads

Session persistence

UI regressions should be detected automatically.

---

# 10.13 AI Validation

AI responses should be verified for:

Scientific consistency

Hallucination prevention

Reference correctness

Prompt compliance

Language quality

Unsupported claims

The AI should never contradict validated metrics.

---

# 10.14 Verification Checklist

Before merging any feature:

- ✓ Unit tests pass
- ✓ Integration tests pass
- ✓ Manual verification completed
- ✓ Logs contain no unexpected errors
- ✓ Database unchanged
- ✓ Existing functionality preserved

Only then is the feature considered complete.

---

#### # 10.15 Continuous Verification

Every implementation phase should include:

Implementation

↓

### Automated Tests

↓

### Manual Verification

↓

### Performance Check

↓

### Documentation Update

↓

Merge

No phase is complete without verification.

---

#### # 10.16 Bug Classification

Critical

Application crash

Scientific corruption

Database corruption

Incorrect scoring

High

Broken workflow

Incorrect report

Video export failure

Medium

Incorrect UI behavior

Minor visualization bugs

Low

Typos

Layout issues

Documentation

Priority always favors scientific correctness over visual appearance.

---

#### # 10.17 Release Checklist

Before every production release:

- ✓ All automated tests pass
- ✓ Manual analysis completed
- ✓ Performance benchmark recorded
- ✓ Database migration verified
- ✓ Logs reviewed
- ✓ Documentation updated
- ✓ Version incremented

---

# 10.18 Success Metrics

Technical

\*\*100\*\*% automated test pass rate

Scientific

Consistent outputs across repeated runs

Performance

Stable execution time

Reliability

Zero crashes

Maintainability

Regression-free updates

---

# 10.19 Engineering Rule

Every bug should result in:

## Root cause analysis

## Permanent fix

## New automated test

This prevents the same issue from returning.

---

# 10.20 Long-Term Vision

Testing is not merely a development activity—it is part of the scientific credibility of SwimAnal

A future release should never rely on developer confidence alone.

It should be supported by measurable evidence that the platform remains:

- Scientifically accurate
- Architecturally stable
- Performance optimized
- Safe for production
- Ready for future expansion

---

## End of Chapter 10

This chapter establishes the official quality assurance framework of SwimAnalyzer AI.

Every future algorithm, AI capability, database feature, and cloud service must be validated thro

> **Engineering Principle #7** > > **If software cannot be deployed reliably, monitored continuo**

---

# 11.1 Purpose

This chapter defines the official deployment strategy for SwimAnalyzer AI.

It covers:

- Local deployment
- Cloud deployment
- Production infrastructure
- Monitoring
- Scaling
- Disaster recovery
- Maintenance

This chapter serves as the operational blueprint for moving from a local desktop application to a

---

# 11.2 Deployment Philosophy

Deployment must evolve gradually.

```  
Developer Machine

↓

Internal Testing

↓

Beta Deployment

↓

Production

↓

Global Platform

Each stage must be fully stable before moving to the next.

11.3 Deployment Roadmap

Stage 1

Local Desktop

Current Status

☒ Completed

Technologies

- Python
 - Streamlit
 - SQLite
 - FFmpeg
 - MediaPipe
-

Stage 2

Private Internal Deployment

Purpose

Coach evaluation

Small athlete group

Bug discovery

Performance measurement

Stage 3

Public Beta

Purpose

Real users

Feedback

Dataset expansion

Scientific validation

Stage 4

Commercial Production

Purpose

Subscription platform

Organizations

Swimming clubs

National teams

11.4 Local Development Environment

Current stack

↓

Streamlit

↓

SQLite

↓

MediaPipe

↓

FFmpeg

Ideal **for**:

Development

Debugging

Scientific validation

Rapid iteration

11.5 Future Production Architecture

``` Internet

↓

### Cloud Load Balancer

↓

Reverse Proxy (Nginx)

↓

FastAPI

↓

### Analysis Workers

↓

PostgreSQL

↓

Redis

↓

### Cloud Storage

↓

Monitoring ```

This architecture supports horizontal scaling.

---

## # 11.6 Containerization

Future production should use Docker.

Example services

``` Frontend

Backend **API**

Worker

Database

Redis

Nginx ```

Benefits

Consistent environments

Easy deployment

Version control

Scalability

11.7 Cloud Infrastructure

Candidate providers

Amazon Web Services (**AWS**)

Microsoft Azure

Google Cloud Platform

DigitalOcean

The software should remain cloud-provider independent.

11.8 File Storage Strategy

Current

``` ### Local Files ```

Future

``` ### Cloud Object Storage

↓

Amazon S3

Azure Blob

Google Cloud Storage

Large videos should never be stored inside the database.

11.9 Database Deployment

Current

SQLite

↓

Future

PostgreSQL

Benefits

Concurrent users

Automatic backup

Replication

High availability

Cloud support

11.10 Background Processing

Long-running video analysis should eventually execute outside the web request.

Future architecture

↓

Queue

↓

Worker

↓

Analysis

↓

Database

↓

Notification ...

Possible technologies

Celery

RQ

Dramatiq

RabbitMQ

Redis Queue

11.11 Monitoring

Every production deployment should monitor:

CPU

Memory

GPU (future)

Disk usage

Database

API latency

Queue length

Error rate

Video processing time

11.12 Logging

Production logs should include

Application startup

User login

Video upload

Analysis started

Analysis completed

Errors

Warnings

Performance metrics

Logs should never contain:

Passwords

API keys

Personal secrets

11.13 Health Checks

Every production service should expose:

``` /health

/status

/version ```

These endpoints verify:

Application availability

Database connection

Model availability

Disk access

Configuration validity

---

# 11.14 Backup Strategy

Database

Daily backup

Weekly archive

Monthly snapshot

Videos

Cloud replication

Version retention

Configuration

Git

Secure backup

Cloud storage

---

# 11.15 Disaster Recovery

Recovery plan

Hardware failure

↓

### Restore Database

↓

### Restore Storage

↓

### Restore Configuration

↓

### Resume Service

Recovery objectives should be documented and tested regularly.

---

#### # 11.16 Security During Deployment

Always use

**\*\*HTTPS\*\***

Encrypted storage

Role-based permissions

Secure authentication

Environment variables

Audit logs

Never deploy secrets inside source code.

---

#### # 11.17 Versioning

Every release should receive a semantic version.

Example

``` 1.0.0

1.1.0

1.2.0

2.0.0 ```

Breaking scientific changes require a major version increment.

11.18 Deployment Checklist

Before production deployment verify:

✓ Automated tests pass

✓ Manual verification complete

- ✓ Configuration validated
- ✓ Database migration tested
- ✓ Logging operational
- ✓ Monitoring enabled
- ✓ Backup configured
- ✓ Documentation updated

11.19 Operational Maintenance

Routine maintenance includes:

Dependency updates

Security patches

Database optimization

Log rotation

Storage cleanup

Performance review

Scientific validation review

Maintenance should be scheduled to minimize user disruption.

11.20 Long-Term Infrastructure Vision

The long-term goal is a globally scalable swimming analysis platform capable of supporting:

- Thousands of simultaneous users
- Millions of stored analyses
- AI-assisted coaching
- Research collaboration
- Enterprise organizations
- National swimming federations

without requiring a redesign of the scientific core.

End of Chapter 11

This chapter defines the deployment and operational strategy of SwimAnalyzer AI.

By following these principles, the platform can evolve confidently from a local Streamlit applica

> **Engineering Principle #8** > > **Users entrust the platform with their personal data and athi**

12.1 Purpose

This chapter defines the official security and privacy framework for SwimAnalyzer AI.

It establishes the rules governing:

- User data protection
- Athlete privacy
- Authentication
- Authorization
- Encryption
- Secure software development
- Regulatory compliance

Every future feature must comply with these principles.

12.2 Security Philosophy

Security is not a feature.

It is a system-wide responsibility.

Every module should assume that:

- Users make mistakes.
- Networks may be compromised.
- Attackers may exist.
- Data has value.

The architecture should therefore follow a **Zero Trust** mindset.

12.3 Security Layers

`` User

↓

Authentication



Authorization



Application



Database



Storage



Backup

Each layer provides independent protection.

12.4 Data Classification

The platform stores several categories of data.

Public Data

Application version

Documentation

General configuration

Internal Data

Application logs

Performance metrics

System configuration

Confidential Data

Athlete profile

Body measurements

Analysis history

Performance reports

Training notes

Sensitive Data

Future authentication credentials

Payment information

Medical notes

Coach comments

Sensitive data receives the highest protection level.

12.5 Authentication

Current Status

Not implemented.

Future authentication should support:

Email

Password

Multi-factor Authentication (**MFA**)

OAuth

Google Sign-In

Microsoft Sign-In

Apple Sign-In

12.6 Authorization

Every user should have clearly defined permissions.

Future roles include:

Administrator

↓

Organization Manager

↓

Coach

↓

Athlete

↓

Viewer

Permissions must always be role-based.

12.7 Password Policy

Passwords should never be stored directly.

Requirements:

Minimum length

Strong complexity

Secure hashing

Salted hashes

Modern password algorithms (e.g., Argon2 or bcrypt)

Plain text passwords are strictly prohibited.

12.8 Encryption

Encryption should protect:

Database backups

Videos

Reports

Cloud storage

****API**** communication

Future cloud deployments should encrypt both:

Data in transit

Data at rest

12.9 API Security

Future APIs should require:

Authentication tokens

****HTTPS****

Rate limiting

Request validation

Audit logging

****API**** keys must never be embedded in source code.

12.10 Secret Management

Never store:

****API**** keys

Passwords

Database credentials

Cloud credentials

Inside:

Git repositories

Configuration files

Source code

Secrets should be managed through secure environment variables or dedicated secret management systems.

12.11 Privacy Principles

Athletes own their data.

The platform only stores data required to provide analysis.

Personal information should never be collected without a valid purpose.

12.12 Data Retention

Future deployments should allow users to:

View stored data

Export their data

Delete their data

Download analysis history

Request account removal

Historical scientific reports should be managed according to the organization's retention policy.

12.13 Logging Policy

Logs should include:

Errors

Warnings

Analysis events

Database operations

Performance statistics

Logs must never contain:

Passwords

Tokens

Private keys

Personal secrets

Sensitive personal information

12.14 Secure File Upload

Every uploaded video should be validated.

Checks include:

Allowed file types

Maximum file size

Corrupted files

Malicious filenames

Unexpected formats

Invalid uploads should be rejected immediately.

12.15 Database Security

Future production databases should enforce:

Role-based access

Encrypted backups

Least privilege

Connection encryption

Automatic backups

Audit trails

12.16 Compliance

Future commercial deployments should be designed with awareness of regulations such as:

****GDPR**** (European Union)

****CCPA**** (California)

Local privacy regulations

Institutional research policies

The platform should minimize personally identifiable information whenever possible.

12.17 Security Monitoring

Production monitoring should detect:

Repeated login failures

Unexpected ****API**** traffic

Database anomalies

Storage failures

Unauthorized access attempts

Critical events should generate alerts.

12.18 Incident Response

If a security incident occurs:

Detect

Contain

Investigate

Recover

Document

Prevent recurrence

Every incident should result in engineering improvements.

12.19 Secure Development Rules

Every new feature must answer:

Does it expose sensitive data?

Does it require authentication?

Does it introduce new attack surfaces?

Can user input be abused?

Are all inputs validated?

If uncertainty exists, perform a security review before implementation.

12.20 Long-Term Security Vision

As SwimAnalyzer AI evolves into a commercial cloud platform, security should become a competitive

The long-term objective is to provide a platform trusted by:

- Individual athletes
- Professional coaches
- Swimming academies
- Universities
- National federations
- Sports science laboratories

through rigorous engineering, transparent privacy practices, and continuous security improvement.

End of Chapter 12

This chapter defines the official security and privacy strategy of SwimAnalyzer AI.

Every future module, cloud service, AI capability, and commercial feature must comply with these

> **Engineering Principle #9** > > **Consistent code is more valuable than clever code.** Every lin

13.1 Purpose

This chapter establishes the official coding standards for SwimAnalyzer AI.

It defines:

- Code organization
- Naming conventions
- Project structure
- Error handling
- Logging
- Design principles
- Refactoring rules
- Feature implementation workflow

Every contributor and every AI coding assistant must follow these standards.

13.2 Development Philosophy

The project follows five core principles:

``` Readable

↓

Maintainable

↓

Testable

↓

Reusable

↓

Scalable ```

Performance is important.

Maintainability is mandatory.

---

# 13.3 Official Code Style

Python standard

\*\*PEP\*\*-8

## Additional rules

- Maximum line length: **\*\*100\*\*** characters
- Explicit imports preferred
- Type hints required
- Meaningful names only
- No abbreviated variable names
- Avoid unnecessary comments

---

### # 13.4 Naming Conventions

#### ## Variables

Good

```
```python stroke_rate body_roll_angle average_visibility completed_cycles ```
```

Bad

```
```python sr br tmp a b x ```
```

---

#### ## Functions

Functions must describe an action.

Good

```
```python calculate_body_roll()
```

```
estimate_stroke_rate()
```

```
validate_video_quality()
```

```
generate_report() ```
```

Bad

```
```python calc()
```

```
do()
```

```
run()
```

```
process() ```
```

---

```
Classes
```

Classes represent nouns.

Good

```
```python AnalysisService
```

VideoProcessor

PoseDetector

AthleteRepository ```

Bad

```
```python Data
```

Manager2

ProcessorFinal ```

```

```

```
Constants
```

Use uppercase.

```
```python MAX_VIDEO_SIZE
```

DEFAULT_FRAME_RATE

MIN_POSE_CONFIDENCE ```

```
---
```

```
# 13.5 Folder Organization
```

Official structure

```
``` app/
```

analysis/

services/

repositories/

models/

database/

core/

utils/

tests/

config/

docs/ ---

Never place business logic inside the UI.

---

### # 13.6 Separation of Concerns

Each layer owns exactly one responsibility.

--- UI

↓

Service

↓

Analysis

↓

Repository

↓

Database ---

Forbidden

--- UI

↓

SQLite ---

Forbidden

--- Analysis

↓

```
Streamlit ```
```

```

```

```
13.7 SOLID Principles
```

```
The project officially adopts SOLID.
```

```
Single Responsibility
```

```
Each class should have one reason to change.
```

```

```

```
Open / Closed
```

```
Extend behavior.
```

```
Avoid modifying stable code.
```

```

```

```
Liskov Substitution
```

```
Every analyzer should be replaceable.
```

```
Example
```

```
``` StrokeAnalyzer
```

```
↓
```

```
FreestyleAnalyzer
```

```
↓
```

```
ButterflyAnalyzer ```
```

```
---
```

```
## Interface Segregation
```

```
Small focused interfaces.
```

```
Avoid giant interfaces.
```

```
---
```

```
## Dependency Inversion
```

Depend on abstractions.

Never on implementations.

13.8 Design Patterns

Official patterns

- ✓ Strategy
- ✓ Repository
- ✓ Factory
- ✓ Dependency Injection
- ✓ Service Layer

Future

Observer

Adapter

Command

State

Builder

13.9 Function Design

A function should ideally:

Do one thing.

Have one responsibility.

Return one clear result.

Preferred

20-40 lines

Avoid functions longer than

100 lines.

13.10 Error Handling

Always catch expected exceptions.

Good

```
```python
try:
 report = exporter.export()
except ExportError:
 logger.exception(*Export failed*)
```

Avoid

```
except:
 pass
```

Silent failures are forbidden.

---

## 13.11 Logging Rules

---

Every important event should be logged.

Required

Application startup

Video upload

Analysis start

Analysis finish

Database save

Warnings

Errors

Do not log:

Passwords

Secrets

Personal identifiers

---

## 13.12 Configuration Rules

---

Hardcoded values are prohibited.

Bad

```
python threshold = 0.65
```

Good

```
python threshold = config.pose.min_confidence
```

Everything configurable belongs in:

```
config.yaml
```

---

## 13.13 Type Hints

---

All public functions require type hints.

Good

```
python def calculate_score(metrics: Metrics) -> float:
```

Bad

```
python def calculate(metrics):
```

---

## 13.14 Documentation

---

Every public class requires:

Purpose

Inputs

Outputs

Exceptions

Example

Docstrings should follow a consistent format.

---

## 13.15 Comments

---

Write comments only when necessary.

Explain **why**.

Not **what**.

Bad

```
python # Increment i i += 1
```

Good

```
python # Ignore unstable frames to reduce pose jitter.
```

---

## 13.16 Refactoring Rules

---

Refactor only when:

Duplication exists

Functions are too long

Responsibilities overlap

Testing becomes difficult

Refactoring must never change scientific behavior.

---

## 13.17 Code Review Checklist

---

Before merging code:

- ✓ Naming is meaningful
  - ✓ No duplicated logic
  - ✓ Tests added
  - ✓ Logging included
  - ✓ Configuration externalized
  - ✓ Documentation updated
  - ✓ Architecture respected
- 

## 13.18 Performance Rules

---

Optimize only after measuring.

Avoid premature optimization.

Measure:

Execution time

Memory

Frame processing

Database queries

---

## 13.19 AI Coding Rules

---

When using AI to generate code:

Always review output.

Never merge AI code without testing.

Require:

Type hints

Logging

Tests

Documentation

Architecture compliance

AI accelerates development.

It never replaces engineering judgment.

---

## 13.20 Feature Development Workflow

---

Every new feature follows this sequence:

↓

Design

↓

### Architecture Review

↓

Implementation

↓

### Unit Tests

↓

### Integration Tests

↓

### Manual Validation

↓

Documentation

↓

Merge ---

Skipping steps is prohibited.

---

# 13.21 Definition of Done

A feature is complete only when:

- ✓ Requirements implemented
- ✓ Tests passing
- ✓ Manual verification complete
- ✓ Logs validated
- ✓ Documentation updated
- ✓ No regressions
- ✓ Architecture respected

Working code alone is **not** sufficient.

---

# 13.22 Technical Debt Policy

Technical debt is acceptable only if:

It is documented.

It has an owner.

It has a planned resolution.

Hidden technical debt is forbidden.

---

# 13.23 Version Control Standards

Every commit should:

Describe one logical change.

Use meaningful messages.

Example

```
``` feat: add athlete repository  
fix: resolve video export race condition  
refactor: split biomechanics calculator  
test: add regression tests for scoring  
docs: update deployment guide ```
```

Avoid vague commits such as:

```
``` update
```

```
fix
```

```
changes
```

```
final
```

```
temp ```
```

```

```

```
13.24 Engineering Rules
```

Before writing code ask:

Does this belong in the correct layer?

Can it be tested?

Can it be reused?

Can another developer understand it?

Can AI safely modify it later?

If the answer is \*No\*, redesign before implementation.

```

```

```
13.25 Long-Term Development Vision
```

SwimAnalyzer AI should remain maintainable for many years.

The codebase should be understandable by:

- New developers
- AI coding assistants

- Researchers
- Coaches with technical backgrounds
- Future engineering teams

The goal is not simply to write code.

The goal is to build a scientific software platform whose quality improves over time rather than

---

## End of Chapter 13

This chapter defines the official development standards for SwimAnalyzer AI.

Every future module, feature, algorithm, AI integration, and infrastructure component must follow

> **Engineering Principle #10** > > **Great software is not built by isolated coding sessions—it**

---

# 14.1 Purpose

This chapter defines the official engineering workflow for SwimAnalyzer AI.

It specifies:

- Development lifecycle
- Branch strategy
- Git workflow
- Code review process
- Release management
- Bug lifecycle
- Documentation workflow
- Decision-making process

Every contributor and every AI coding assistant must follow this workflow.

---

# 14.2 Engineering Philosophy

Every engineering task follows the same lifecycle.

``` Requirement

↓

Design

↓

Implementation

↓

Testing

↓

Review

↓

Documentation

↓

Release ```

Skipping any stage is prohibited.

14.3 Development Lifecycle

Every feature passes through seven phases.

Phase 1

Requirements

Questions answered:

- Why is this needed?
- Who benefits?
- What problem is solved?
- What scientific impact exists?

Phase 2

Architecture

Determine:

- Responsible layer
- Services
- Database impact
- AI impact
- UI impact

No code is written during this phase.

Phase 3

Implementation

Only after architecture approval.

Implementation must follow:

- **SOLID**
- Repository Pattern
- Strategy Pattern
- Configuration rules

Phase 4

Testing

Required:

- Unit tests
- Integration tests
- Manual verification
- Performance verification

Phase 5

Review

Code review.

Architecture review.

Scientific review.

Phase 6

Documentation

Update:

Engineering Playbook

****README****

****API**** documentation

Developer notes

Phase 7

Release

Only after all previous phases succeed.

14.4 Git Workflow

Official branching strategy

``` main

↓

develop

↓

feature/\*

↓

bugfix/\*

↓

hotfix/\* ```

---

# 14.5 Branch Definitions

## main

Production-ready code only.

Protected branch.

No direct commits.

---

## develop

Integration branch.

Receives completed features.

---

## feature

Example

``` feature/athlete-dashboard

feature/plotly-progress

feature/ai-coach ```

One feature per branch.

bugfix

Example

``` bugfix/video-export

bugfix/session-storage ```

---

## hotfix

Emergency production fixes.

Merged directly into production.

---

# 14.6 Commit Standards

Every commit should represent one logical change.

Preferred format

``` feat:

fix:

refactor:

test:

docs:

perf:

style:

chore: ```

Examples

``` feat: add athlete comparison page

fix: resolve duplicate pose inference

refactor: split biomechanics calculator

docs: update engineering playbook

test: add repository tests ```

---

# 14.7 Pull Request Workflow

Every Pull Request should include:

Purpose

Affected modules

Screenshots (if UI)

Test evidence

Performance impact

Architecture impact

Documentation updates

A Pull Request without context should never be approved.

---

# 14.8 Code Review Checklist

Reviewer verifies:

- ✓ Architecture respected
- ✓ Naming conventions
- ✓ No duplicated logic
- ✓ Tests added
- ✓ Logging added
- ✓ Documentation updated
- ✓ No unnecessary complexity
- ✓ Scientific correctness preserved

---

#### # 14.9 Bug Lifecycle

Every bug follows the same lifecycle.

``` Reported

↓

Confirmed

↓

Prioritized

↓

Assigned

↓

Implemented

↓

Verified

↓

Closed ```

Every closed bug should include a root cause.

14.10 Severity Levels

Critical

Application crash

Scientific corruption

Database corruption

High

Broken workflow

Incorrect calculations

Major UI failure

Medium

Unexpected behavior

Minor logic issues

Low

Visual defects

Documentation

Typos

14.11 Feature Request Workflow

Every new feature begins with:

Problem statement

↓

Scientific justification

↓

Architecture review

↓

Implementation plan

↓

Development

↓

Validation

↓

Documentation

14.12 Scientific Review

Any change affecting:

Performance score

Stroke metrics

Joint angles

Confidence

Consistency validation

requires scientific review before approval.

14.13 Documentation Workflow

Documentation is updated whenever:

Architecture changes

Database changes

Algorithms change

AI behavior changes

Deployment changes

Documentation always evolves with the software.

14.14 Release Workflow

Release sequence

```  
### Feature Freeze

↓

### Regression Tests

↓

### Performance Benchmark

↓

### Documentation Review

↓

### Version Tag

↓

### Production Release

No new features during Feature Freeze.

---

## 14.15 Version Management

---

### Semantic Versioning

```
MAJOR.**MINOR**.**PATCH**
```

Example

1.1.0

1.1.1

2.0.0 ---

Major versions indicate architectural or scientific changes.

---

# 14.16 Decision Making

Engineering decisions should prioritize:

Scientific correctness

↓

Maintainability

↓

Scalability

↓

Performance

↓

Convenience

Convenience should never override architecture.

---

# 14.17 Technical Debt Workflow

When technical debt is accepted:

Record it.

Assign ownership.

Define resolution date.

Track progress.

Technical debt without documentation is prohibited.

---

#### # 14.18 AI-Assisted Development

AI may assist with:

Code generation

Refactoring

Documentation

Testing

Architecture suggestions

AI may never:

Approve its own code

Skip testing

Replace engineering review

Override scientific validation

---

#### # 14.19 Sprint Workflow

Typical sprint

``` Planning

↓

Implementation

↓

Testing

↓

Review

↓

Documentation

↓

Release

↓

Retrospective ---

Every sprint should improve both the product and the engineering process.

14.20 Continuous Improvement

After every completed phase ask:

What worked?

What failed?

What slowed development?

What can be automated?

Engineering maturity grows through continuous refinement.

14.21 Engineering Governance

The Engineering Playbook is the single source of truth.

If implementation conflicts with the Playbook:

The implementation must be corrected.

The Playbook is updated only through deliberate engineering decisions.

14.22 Long-Term Workflow Vision

As SwimAnalyzer AI grows into a commercial platform, this workflow should support:

- Individual developers
- Distributed engineering teams
- AI coding assistants
- Scientific collaborators
- External contributors
- Enterprise development

without sacrificing quality, consistency, or scientific integrity.

End of Chapter 14

This chapter defines the official engineering workflow and governance model for SwimAnalyzer AI.

Every feature, bug fix, scientific enhancement, and AI-generated contribution must follow this workflow.

> **Scientific Principle #2** > > Every algorithm in SwimAnalyzer AI must be deterministic, explainable, and reproducible.

15.1 Purpose

This chapter serves as the official scientific specification of every algorithm used within SwimAnalyzer AI.

It defines:

- Algorithm responsibilities
- Inputs
- Outputs
- Mathematical assumptions
- Scientific limitations
- Validation strategy

This document is the reference for both developers and researchers.

15.2 Scientific Pipeline

Every analysis follows exactly the same pipeline.

``` Video

↓

### Video Validation

↓

### Frame Extraction

↓

### Pose Detection

↓

### Landmark Filtering



### Stroke Phase Detection



### Biomechanical Measurements



### Scientific Validation



### Performance Scoring



### Report Generation

No stage may be skipped.

---

## 15.3 Video Validation Algorithm

---

### Objective

---

Determine whether the uploaded video is suitable for scientific analysis.

### Inputs

Video file

### Outputs

### Video Quality Report

### Measurements

- Resolution
- **FPS**
- Brightness

- Blur
- Camera stability
- Pose visibility
- Occlusion
- Frame completeness

## Possible Results

High

Medium

Low

Critical

Critical videos should not produce scientific conclusions.

---

# 15.4 Pose Detection

---

## Current Engine

MediaPipe Pose Landmarker

## Input

RGB frame

## Output

33 body landmarks

Each landmark contains:

y

z

visibility

presence ...

The detector is treated as an external dependency.

Its outputs are validated before further processing.

---

#### # 15.5 Landmark Validation

Before biomechanical calculations:

Verify visibility.

Reject unstable landmarks.

Smooth temporal jitter.

Handle missing landmarks.

Estimate confidence.

Low-confidence landmarks reduce scientific confidence.

They never increase performance scores.

---

#### # 15.6 Stroke Phase Detection

Current implementation

Freestyle only.

Target phases

Recovery

↓

Entry

↓

Catch

↓

Pull

↓

Push



↓

## Recovery

A complete cycle requires all phases in the correct order.

Future versions may use probabilistic state estimation.

---

## # 15.7 Stroke Cycle Detection

### Objective

Count complete swimming cycles.

### Inputs

Phase sequence

### Outputs

Completed cycles

Incomplete cycles

Cycle duration

### Reliability

Cycle detection affects confidence but should not directly modify technique quality.

---

## # 15.8 Joint Angle Calculation

Angles are calculated using landmark geometry.

### Examples

Elbow

Shoulder

Hip

Knee

The calculations must be:

Deterministic

Repeatable

Frame-independent

Future improvements may include temporal smoothing.

---

# 15.9 Body Roll Estimation

Current approach

Estimated from shoulder and torso orientation.

Future improvements

3D estimation

Multi-camera calibration

\*\*IMU\*\* fusion

Body roll should always indicate confidence.

---

# 15.10 Stroke Rate

Definition

Completed cycles per minute.

Formula

``` ### Stroke Rate

=

Completed Cycles

×

60

/

Analysis Duration

Output

SPM

(Stroke Per Minute)

15.11 Stroke Length

Current implementation

Relative estimation.

Depends on:

Calibration

Pose tracking

Frame consistency

Future implementation

Anthropometric calibration

Real-world metric distance

15.12 Kick Frequency

Estimated using:

Hip landmarks

Knee landmarks

Ankle landmarks

Future versions should distinguish:

Flutter kick

Dolphin kick

Breaststroke kick

15.13 Symmetry Evaluation

Objective

Compare left and right body movement.

Examples

Arm timing

Stroke path

Joint angles

Recovery timing

Future versions may compare elite references.

15.14 Calibration Engine

Current

Relative calibration

Future

User body measurements

↓

Metric conversion

↓

Higher accuracy

Calibration uncertainty should always be reported.

15.15 Scientific Consistency Validator

Purpose

Detect contradictions.

Examples

Excellent score

- Critical video quality

↓

Invalid

Example

Zero stroke cycles

- Excellent efficiency

↓

Contradiction

The validator protects scientific credibility.

15.16 Performance Score

Current score combines

Technique

↓

Consistency

↓

Biomechanics

↓

Video quality influence

Future versions

Elite benchmark weighting

Age normalization

Gender normalization

Confidence calibration

15.17 Confidence Score

Separate from performance.

Confidence depends on

Video quality

Pose stability

Completed cycles

Landmark visibility

Calibration quality

Future benchmark availability

High confidence does not necessarily imply high performance.

15.18 Scientific Rules

Every algorithm must satisfy:

Deterministic

Explainable

Traceable

Testable

Configurable

Algorithms violating these principles should not be included.

15.19 Error Detection

Current detectable faults

Early hand exit

Low elbow

Poor symmetry

Incomplete cycles

Poor body roll

Future faults

Overreaching

Dropped hips

Cross-over entry

Late catch

Excessive kick

Head instability

15.20 Algorithm Dependencies

↓

Stroke Detection

↓

Biomechanics

↓

Consistency

↓

No downstream algorithm should modify upstream measurements.

15.21 Validation Strategy

Every algorithm must include

Unit tests



Integration tests



Scientific verification



Regression tests



Documentation

An undocumented algorithm is considered incomplete.

15.22 Future Research Expansion

Future algorithm improvements include:

Machine-learning stroke segmentation

Elite reference datasets

Automatic anomaly detection

3D pose estimation

Multi-camera reconstruction

Wearable sensor fusion

Competition analysis

Research-grade benchmarking

15.23 Scientific Integrity Rule

If an algorithm cannot explain:

- What it measures
- How it measures
- Why it matters
- How reliable it is

then it must not be included in the platform.

15.24 Long-Term Scientific Vision

The scientific engine should become the most reliable and extensible swimming biomechanics engine available.

Its design should allow future researchers to:

- Replace algorithms
- Improve calculations
- Validate new metrics
- Publish reproducible studies

without changing the surrounding platform architecture.

End of Chapter 15

This chapter defines the official specification of the scientific algorithms used by SwimAnalyzer AI.

Every future calculation, metric, and biomechanics module must comply with these specifications to preserve scientific validity, reproducibility, and long-term maintainability.

AI Engineering Principle > > An AI model is only as effective as the engineering process that guides it. Prompts should be treated as reusable engineering assets—not disposable conversations.

16.1 Purpose

This chapter defines the official Prompt Engineering Library for SwimAnalyzer AI.

Every AI interaction should be:

- Repeatable
- Structured
- Context-aware
- Architecture-aware
- Scientifically accurate

These prompts are designed to work with:

- ChatGPT
 - Claude
 - Gemini
 - DeepSeek
 - Qwen
 - Llama
 - Future LLMs
-

16.2 General Prompt Rules

Every prompt should include five sections:

↓

Context

↓

Objective

↓

Constraints

↓

Expected Output

This structure dramatically improves response quality.

16.3 Master System Prompt (Use at the Beginning of Every New AI Session)

Your responsibilities include:

- Preserving scientific correctness.
- Preserving the existing architecture.
- Following ****SOLID**** principles.
- Respecting the Repository Pattern.
- Respecting the Strategy Pattern.
- Never breaking backward compatibility.
- Never introducing unnecessary complexity.
- Never modifying stable business logic unless explicitly instructed.

Always explain architectural decisions before implementation.

Whenever possible:

- Prefer maintainability over cleverness.
- Prefer modularity over shortcuts.
- Prefer reusable code over duplication.

Assume this project will become a commercial AI platform used by swimming academies, researchers

16.4 Architecture Review Prompt

``` Act as a Principal Software Architect.

Review the following implementation.

Evaluate:

- **\*\*SOLID\*\*** compliance
- Separation of Concerns
- Design Patterns
- Scalability
- Maintainability
- Future extensibility

Do **\*\*NOT\*\*** modify code.

Produce only an engineering audit. ```

---

# 16.5 Refactoring Prompt

``` Refactor the provided implementation while preserving **\*\*100\*\***% of existing behavior.

Requirements:

- No feature changes.
- No UI changes.
- No ****API**** changes.
- Improve readability.
- Reduce duplication.
- Increase modularity.
- Preserve all tests.
- Explain every architectural decision.

16.6 Bug Fix Prompt

Identify the root cause of the issue.

Do not propose temporary fixes.

Provide:

Root Cause

Impact Analysis ## Permanent Solution ## Regression Risk ## Verification Plan ```

16.7 Performance Optimization Prompt

``` Analyze the system for performance bottlenecks.

Focus on:

**CPU**

Memory

Video processing

Database

MediaPipe

Streamlit

Recommend only measurable optimizations.

Never sacrifice correctness. ```

---

# 16.8 Scientific Validation Prompt

``` Review the biomechanics implementation.

Evaluate:

Scientific validity

Mathematical correctness

Biomechanics assumptions

Potential bias

Measurement uncertainty

Confidence estimation

Do not review coding style.

Review scientific quality only. ```

16.9 AI Coach Prompt

``` Act as an Olympic-level swimming coach.

Explain the athlete's report.

#### Requirements:

Use simple language.

Never invent metrics.

Never contradict measured data.

#### Separate:

Observed facts

Interpretation

Recommendations

Future training priorities. ```

---

# 16.10 UI Review Prompt

``` Review the user interface.

Evaluate:

Navigation

Accessibility

Clarity

Coach usability

Athlete usability

Information hierarchy

Professional appearance

Do not rewrite code.

Produce UX recommendations only. ```

16.11 Database Review Prompt

``` Act as a Database Architect.

Review the persistence layer.

Evaluate:

Normalization

Relationships

Indexes

Scalability

### Repository Pattern

Future PostgreSQL migration

No implementation changes. ```

---

# 16.12 Security Review Prompt

``` Perform a security audit.

Evaluate:

Authentication

Authorization

Secrets

Input validation

SQL Injection

File Upload

Logging

Privacy

Compliance

Return a prioritized report. ```

16.13 Documentation Prompt

``` Write professional documentation.

#### Audience:

Software engineers

Researchers

Future contributors

#### Requirements:

Clear

Technical

Structured

Accurate

No marketing language. ```

---

# 16.14 Code Generation Prompt

``` Generate production-ready code.

Requirements:

PEP-8

Type hints

Logging

Configuration driven

Unit-test friendly

No magic numbers

No duplicated logic

Architecture compliant ```

16.15 Testing Prompt

``` Generate comprehensive tests.

#### Include:



```
Unit Tests

Integration Tests

Edge Cases

Failure Cases

Regression Tests

Performance Validation

Scientific Validation
```

---

## 16.16 API Design Prompt

---

Requirements:

RESTful

Versioned

Documented

Secure

Scalable

Stateless

Return OpenAPI-compatible endpoints. ```

---

# 16.17 Deployment Prompt

``` Design a production deployment.

Include:

Docker

Nginx

FastAPI

PostgreSQL

Redis

Monitoring

Logging

Backup

Security

CI/CD ```

16.18 Research Prompt

``` Search scientific literature.

Return only peer-reviewed evidence.

Include:

\*\*DOI\*\*

Year

Authors

Journal

Methodology

Limitations

Relevance to swimming biomechanics. ```

---

# 16.19 Feature Design Prompt

``` Design a new feature.

Return:

Business justification

Architecture

Database impact

Scientific impact

UI impact

Testing strategy

Deployment impact

Do not implement. ```

16.20 AI Code Review Prompt

``` Review generated code.

Evaluate:

Correctness

Architecture

Readability

Maintainability

Performance

Security

Scientific integrity

Assign a score from 0-100. ```

---

# 16.21 Prompt for New Swimming Stroke

``` Design support for a new swimming stroke.

Requirements:

Reuse existing architecture.

Extend Strategy Pattern.

Avoid modifying Freestyle implementation.

Generate only the required new modules. ```

16.22 Prompt for New AI Model

``` Evaluate whether a new AI model should be integrated.

Analyze:

Accuracy

Latency

Cost

Hardware requirements

Scientific benefits

Deployment impact

Migration strategy ```

---

# 16.23 Prompt for Large Refactoring

``` Plan a large architectural refactor.

Return:

Affected modules

Migration order

Risk assessment

Rollback strategy

Verification plan

Do not modify code. ```

16.24 Prompt for Release Audit

``` Conduct a pre-release engineering audit.

Evaluate:

Architecture

Performance

Security

Documentation

Scientific correctness

Testing

Deployment readiness

Return **PASS** or **FAIL** with justification. ```

---

# 16.25 Prompt Engineering Rules

Every prompt should:

- ✓ Assign a role
- ✓ Define context
- ✓ Define constraints
- ✓ Define output format
- ✓ Preserve architecture
- ✓ Preserve scientific correctness

Avoid vague requests.

---

# 16.26 AI Collaboration Workflow

Every AI-assisted task should follow:

``` Planning

↓

Architecture Review

↓

Implementation

↓

Testing

↓

Verification

↓

Documentation ```

Never skip architecture review.

16.27 Prompt Versioning

Prompts should be version-controlled just like source code.

Example:

``` Prompt v1.0

↓

Prompt v1.1

↓

Prompt v2.0 ```

Document why each prompt changed.

---

# 16.28 Long-Term AI Vision

As SwimAnalyzer AI grows, the Prompt Library should become a reusable engineering framework capable of

The objective is not merely to generate code, but to establish a standardized AI-assisted development process.

- Scientific accuracy
- Architectural integrity
- Code quality
- Long-term maintainability

This Prompt Library is therefore considered part of the project's core engineering assets.

---

## End of Chapter 16

This chapter establishes the official AI Prompt Engineering Library for SwimAnalyzer AI.

Every future interaction with an AI coding assistant should be based on these standardized prompts

> **Engineering Principle #11** > > **Every production deployment should follow a checklist-not a**

---

# 17.1 Purpose

This chapter defines the official production checklists for SwimAnalyzer AI.

Every engineering activity must be verified using standardized checklists before moving to the next

These checklists reduce:

- Human error
- Regression
- Scientific inconsistency
- Production failures

---

# 17.2 Development Checklist

Before writing any code:

- ☐ Requirement understood
- ☐ Scientific objective identified
- ☐ Architecture reviewed
- ☐ Correct project layer selected
- ☐ Existing implementation searched
- ☐ Configuration requirements identified
- ☐ Test strategy prepared

Only after completing this checklist should implementation begin.

---

# 17.3 Feature Implementation Checklist

Before starting a new feature:

- ☐ Business justification documented
- ☐ Architecture impact analyzed
- ☐ Database impact analyzed
- ☐ UI impact analyzed
- ☐ AI impact analyzed
- ☐ Scientific impact analyzed
- ☐ Future scalability considered

No feature should be implemented without a clear purpose.

---

#### # 17.4 Coding Checklist

Before committing code:

- ☐ Follows **\*\*PEP\*\***-8
- ☐ Uses type hints
- ☐ No duplicated logic
- ☐ No magic numbers
- ☐ Logging added
- ☐ Exceptions handled
- ☐ Configuration externalized
- ☐ Documentation updated
- ☐ Meaningful variable names
- ☐ **\*\*SOLID\*\*** respected

---

#### # 17.5 Scientific Algorithm Checklist

Before merging a new algorithm:

- ☐ Scientific purpose defined



☐ Mathematical assumptions documented

☐ Inputs documented

☐ Outputs documented

☐ Confidence estimation included

☐ Limitations documented

☐ Unit tests added

☐ Regression tests added

☐ Scientific review completed

---

#### # 17.6 Database Checklist

Before changing persistence:

☐ Migration reviewed

☐ Foreign keys verified

☐ Relationships validated

☐ Repository updated

☐ Tests added

☐ Rollback strategy prepared

☐ Backup verified

---

#### # 17.7 User Interface Checklist

Before releasing UI changes:

☐ Navigation unchanged

☐ Accessibility preserved

☐ Responsive layout verified

☐ Error messages clear

☐ Loading indicators added

☐ Empty states handled

☐ Charts verified

☐ Session persistence verified

---

#### # 17.8 AI Feature Checklist

Before integrating AI:

☐ Uses validated scientific data

☐ No hallucination risk

☐ Prompt reviewed

☐ Scientific references preserved

☐ Confidence respected

☐ Safety review completed

☐ Output explainable

AI should never become the source of truth.

---

#### # 17.9 Performance Checklist

Before release:

☐ \*\*CPU\*\* usage measured

☐ Memory usage measured

☐ Analysis time benchmarked

☐ Database performance verified

☐ Video export benchmarked

☐ Large video tested

☐ Long session tested

Optimization should always be evidence-based.

---

#### # 17.10 Security Checklist

##### Before deployment:

- ☐ Secrets removed
- ☐ Environment variables configured
- ☐ \*\*HTTPS\*\* enabled
- ☐ Input validation verified
- ☐ File upload protected
- ☐ Logging sanitized
- ☐ Backup configured
- ☐ Permissions reviewed

---

#### # 17.11 Testing Checklist

##### Before merge:

- ☐ Unit tests pass
- ☐ Integration tests pass
- ☐ Regression tests pass
- ☐ Manual verification completed
- ☐ Edge cases tested
- ☐ Performance verified
- ☐ Scientific validation completed

No feature is complete without successful verification.

---

#### # 17.12 Documentation Checklist

##### Before release:

☐ **\*\*README\*\*** updated

☐ Engineering Playbook updated

☐ Configuration documented

☐ **\*\*API\*\*** documented

☐ Database documented

☐ Architecture diagrams updated

Documentation is part of the product.

---

#### # 17.13 Release Checklist

Before creating a new version:

☐ All tests passing

☐ Performance benchmark recorded

☐ Security review complete

☐ Documentation updated

☐ Version incremented

☐ Release notes prepared

☐ Backup created

☐ Production configuration verified

---

#### # 17.14 Deployment Checklist

Before deploying:

☐ Database available

☐ Models verified

☐ Static files verified

☐ Logging operational

☐ Monitoring operational

- ☐ Storage writable
- ☐ Configuration correct
- ☐ Health check passes

Deployment should never be performed blindly.

---

#### # 17.15 Post-Deployment Checklist

Immediately after deployment:

- ☐ Application starts
- ☐ Login works (future)
- ☐ Video upload works
- ☐ Analysis completes
- ☐ Database writes succeed
- ☐ Reports generated
- ☐ Charts render
- ☐ Logs clean
- ☐ Monitoring healthy

---

#### # 17.16 Bug Resolution Checklist

Before closing a bug:

- ☐ Root cause identified
- ☐ Permanent fix implemented
- ☐ Regression test added
- ☐ Manual verification completed
- ☐ Documentation updated (if needed)

Every resolved bug strengthens the platform.

---

#### # 17.17 Refactoring Checklist

Before approving a refactor:

- ☐ Behavior unchanged
- ☐ Tests unchanged
- ☐ Performance measured
- ☐ Architecture improved
- ☐ Complexity reduced
- ☐ Documentation updated

Refactoring should improve maintainability without changing scientific outputs.

---

#### # 17.18 Production Readiness Checklist

A release is production-ready only if:

- ✓ Zero known critical bugs
- ✓ Scientific validation complete
- ✓ Performance benchmark acceptable
- ✓ Security review passed
- ✓ Documentation complete
- ✓ Backup strategy verified
- ✓ Monitoring active
- ✓ Rollback strategy prepared

---

#### # 17.19 Engineering Decision Checklist

Before making any architectural decision ask:

Does it improve maintainability?

Does it improve scalability?

Does it preserve scientific correctness?

Can it be tested?

Can future developers understand it?

Can future AI systems extend it safely?

If any answer is **No**, reconsider the design.

---

## # 17.20 Long-Term Operational Vision

The goal is to make every deployment predictable, repeatable, and low-risk.

As SwimAnalyzer AI grows into a commercial platform, these checklists become the operational star

Following them consistently ensures that quality is maintained regardless of project size or tear

---

## ## End of Chapter 17

This chapter establishes the official operational checklists for SwimAnalyzer AI.

Every implementation, release, deployment, and production update must follow these checklists to

> **Vision Principle** > > **SwimAnalyzer AI is not intended to become another video analysis app**

---

## # 18.1 Purpose

This chapter defines the long-term vision of SwimAnalyzer AI.

It describes:

- Product evolution
- Scientific evolution
- AI evolution
- Business strategy
- Global expansion
- Research direction

Every engineering decision should support this long-term vision.

---

## # 18.2 Vision Statement

**\*\*To become the global reference platform for AI-assisted swimming performance analysis, combining**

---

### # 18.3 Mission

Our mission is to make elite-level swimming analysis available to every swimmer, regardless of ge

The platform should democratize sports science through artificial intelligence.

---

### # 18.4 Product Evolution Roadmap

The product evolves through five stages.

``` ### Scientific Analyzer

↓

Coaching Platform

↓

Training Management System

↓

Sports Intelligence Platform

↓

Global AI Ecosystem ```

Each stage builds upon the previous one.

18.5 Current Stage

Current capabilities include:

- ✓ Freestyle analysis
- ✓ Pose detection
- ✓ Video quality assessment
- ✓ Athlete profiles

✓ Session history

✓ Interactive dashboards

✓ Scientific reporting

This forms the foundation of the platform.

18.6 Stage Two – Multi-Stroke Support

Expand support to:

- Backstroke
- Breaststroke
- Butterfly
- Individual Medley

Each stroke should reuse the existing Strategy Pattern.

No duplication of business logic.

18.7 Stage Three – AI Coach

Introduce an AI Coach capable of:

- Explaining reports
- Identifying weaknesses
- Suggesting drills
- Tracking progress
- Answering athlete questions

The AI Coach should **interpret** validated data—not generate measurements.

18.8 Stage Four – Benchmark Intelligence

Build a scientific benchmark database.

Reference groups may include:

- Age categories
- Gender
- Skill levels
- Elite swimmers
- National teams

Every analysis can then be compared against relevant reference populations.

18.9 Stage Five – Personalized Training

The platform generates:

- Weekly plans
- Monthly plans
- Technique priorities
- Recovery recommendations
- Progress milestones

Training becomes data-driven rather than subjective.

18.10 Stage Six – Research Platform

Future research capabilities include:

- Anonymous dataset export
- Population studies
- Scientific dashboards
- University collaboration
- Clinical biomechanics

The platform should support reproducible research.

18.11 Multi-Sport Expansion

Although designed for swimming, the architecture should eventually support:

- Running
- Cycling
- Rowing
- Diving
- Triathlon
- Rehabilitation

The analysis engine remains modular.

18.12 Commercial Strategy

Target customers include:

Individual Athletes

Personal performance analysis.

Coaches

Team management.

Technique monitoring.

Progress tracking.

Swimming Clubs

Centralized athlete database.

Coach collaboration.

Season planning.

National Federations

Talent identification.

Competition analysis.

National performance monitoring.

Universities

Research.

Biomechanics.

Sports science education.

18.13 Revenue Model

Potential revenue streams include:

- Subscription plans

- Team licenses
- Federation licenses
- Enterprise solutions
- Research partnerships
- **API** licensing
- Professional consulting

Scientific credibility remains the primary competitive advantage.

18.14 Technology Evolution

Current stack:

- Streamlit
- SQLite
- MediaPipe
- Plotly

Future stack:

- React
- FastAPI
- PostgreSQL
- Redis
- Docker
- Kubernetes
- Cloud AI services

Technology may evolve.

Scientific architecture should remain stable.

18.15 AI Evolution

Current AI role:

Interpretation.

Future AI capabilities:

- Automated coaching
- Technique prediction
- Injury risk estimation
- Training optimization
- Performance forecasting
- Natural language interaction

AI always operates on validated scientific data.

18.16 Scientific Evolution

Future scientific improvements include:

- 3D pose estimation
- Multi-camera reconstruction
- Wearable sensor integration
- Automatic calibration
- Hydrodynamic estimation
- Competition analytics

Every enhancement should undergo scientific validation before production.

18.17 Global Expansion Strategy

Initial deployment:

Individual users.

↓

Regional clubs.

↓

National organizations.

↓

International federations.

↓

Global platform.

Localization should support multiple languages while preserving scientific terminology.

18.18 Engineering Growth

As the platform grows, engineering should evolve from:

Single Developer

↓

Small Engineering Team

↓

Specialized Teams

- AI
- Backend
- Frontend
- Scientific Research
- DevOps
- QA

The Engineering Playbook becomes the shared reference for all teams.

18.19 Success Metrics

Technical

- System availability
- Analysis speed
- Test coverage
- Deployment reliability

Scientific

- Accuracy
- Repeatability
- Peer-reviewed validation
- Benchmark quality

Business

- Active users
- Organizations
- Subscription retention
- Customer satisfaction

Research

- Published studies
- Academic collaborations
- Benchmark datasets

18.20 Ultimate Vision

The long-term ambition is for SwimAnalyzer AI to become:

- The standard platform used by swimming academies.
- A trusted scientific tool for biomechanics researchers.
- A daily coaching assistant for professional coaches.
- A personalized AI mentor for athletes.
- A global reference for swimming performance analysis.

The platform should combine engineering excellence, scientific rigor, and artificial intelligence

End of Chapter 18

This chapter defines the strategic future of SwimAnalyzer AI.

Every architectural decision, scientific enhancement, AI capability, and commercial initiative sh

```markdown id=\*engineering-playbook-ch19\* # Chapter 19 # Engineering Golden Rules

> **Engineering Principle #12** > > **Technology changes. Frameworks evolve. AI models improve. E**

---

# 19.1 Purpose

This chapter establishes the permanent engineering principles that every developer, researcher, /

Unlike implementation details, these rules are intended to remain stable for the lifetime of the

---

# 19.2 Architecture Rules

### Rule 1

Business logic never belongs in the UI.

---

### Rule 2

The UI may display data but must never calculate scientific metrics.

---

### Rule 3

Every layer has exactly one responsibility.

---

#### ### Rule 4

Services coordinate logic. They do not own scientific algorithms.

---

#### ### Rule 5

Repositories communicate with storage only.

---

#### ### Rule 6

Scientific calculations must remain independent of the database.

---

#### ### Rule 7

Every architecture decision should reduce coupling.

---

#### ### Rule 8

Favor composition over inheritance whenever possible.

---

#### ### Rule 9

Never duplicate business logic.

---

#### ### Rule 10

If logic exists twice, it belongs in a shared component.

---

### # 19.3 Scientific Rules

#### ### Rule 11

Never invent scientific data.

---



### Rule 12

Every reported metric must originate from measurable evidence.

---

### Rule 13

Confidence is not performance.

---

### Rule 14

Video quality affects confidence-not athlete ability.

---

### Rule 15

Scientific uncertainty must always be communicated.

---

### Rule 16

Missing information should reduce confidence, never inflate scores.

---

### Rule 17

Every equation must be explainable.

---

### Rule 18

Every algorithm must be reproducible.

---

### Rule 19

Every calculation must be independently testable.

---

### Rule 20

Scientific integrity always overrides marketing.

---

# 19.4 AI Rules

### Rule 21

AI explains. It does not fabricate.

---

### Rule 22

AI recommendations must reference measured data.

---

### Rule 23

AI must never hide uncertainty.

---

### Rule 24

Never allow AI to modify scientific measurements.

---

### Rule 25

Every AI response should distinguish:

### Observed Facts

↓

Interpretation

↓

Recommendations

---

### Rule 26

AI outputs must remain auditable.

---

### Rule 27

Prompt quality determines output quality.

---

### Rule 28

AI-generated code requires human review.

---

### Rule 29

AI accelerates engineering. It does not replace engineering.

---

### Rule 30

Every AI integration must improve user value.

---

# 19.5 Code Quality Rules

### Rule 31

Readable code is more valuable than clever code.

---

### Rule 32

Optimize only after measurement.

---

### Rule 33

Never introduce unnecessary complexity.

---

### Rule 34

Every public function requires documentation.

---

### Rule 35

Every public function requires type hints.

---

### Rule 36

Meaningful names are mandatory.

---

### Rule 37

Magic numbers are prohibited.

---

### Rule 38

Configuration belongs in configuration files.

---

### Rule 39

Logging is mandatory for critical operations.

---

### Rule 40

Silent failures are forbidden.

---

# 19.6 Testing Rules

### Rule 41

Every bug requires a regression test.

---

### Rule 42

Untested code is incomplete.

---

### Rule 43

Every algorithm requires unit tests.

---

### Rule 44

Every workflow requires integration tests.

---

### Rule 45

Manual verification complements automation.

---

### Rule 46

Performance should be benchmarked before optimization.

---

### Rule 47

Never merge failing tests.

---

### Rule 48

Deterministic outputs are preferred whenever possible.

---

### Rule 49

Every scientific update requires validation.

---

### Rule 50

Testing protects the future.

---

# 19.7 Database Rules

### Rule 51

The domain layer never depends on **\*\*SQL\*\***.

---

### Rule 52

Repositories isolate persistence.

---

### Rule 53

Database migrations must be reversible.

---

### Rule 54

Never expose database internals to the UI.

---

### Rule 55

Backups are mandatory before schema changes.

---

### Rule 56

Normalize data unless denormalization is justified.

---

### Rule 57

Indexes should be evidence-driven.

---

### Rule 58

Protect data integrity before optimizing performance.

---

### Rule 59

Data ownership must be explicit.

---

### Rule 60

Every persistent object requires a unique identifier.

---

# 19.8 Security Rules

### Rule 61

Never trust user input.

---

### Rule 62

Validate everything.

---

### Rule 63

Escape everything necessary.

---

### Rule 64

Secrets never belong in source code.

---

### Rule 65

Authentication precedes authorization.

---

### Rule 66

Least privilege is the default.

---

### Rule 67

Every uploaded file is untrusted.

---

### Rule 68

Logs must never expose sensitive information.

---

### Rule 69

**\*\*HTTPS\*\*** is mandatory in production.

---

### Rule 70

Privacy is a design requirement—not an afterthought.

---

# 19.9 Deployment Rules

### Rule 71

Every deployment must be reversible.

---

### Rule 72

Never deploy untested code.

---

### Rule 73

Monitor every production release.

---

### Rule 74

Measure before scaling.

---

### Rule 75

Automate repetitive deployment tasks.

---

### Rule 76



Configuration differs from code.

---

### Rule 77

Infrastructure should be reproducible.

---

### Rule 78

Production should never be used for experimentation.

---

### Rule 79

Every release requires release notes.

---

### Rule 80

Availability is a feature.

---

# 19.10 Engineering Culture Rules

### Rule 81

Solve root causes—not symptoms.

---

### Rule 82

Prefer simplicity over cleverness.

---

### Rule 83

Every improvement should reduce future work.

---

### Rule 84

Document important decisions.

---

### Rule 85

Question assumptions regularly.

---

### Rule 86

Respect scientific evidence.

---

### Rule 87

Respect future developers.

---

### Rule 88

Code for maintainability.

---

### Rule 89

Never stop refactoring responsibly.

---

### Rule 90

Continuous improvement is part of engineering.

---

# 19.11 Product Rules

### Rule 91

Every feature must solve a real problem.

---

### Rule 92

Features never compromise scientific quality.

---

### Rule 93

User trust is more valuable than feature count.

---

### Rule 94

Professional UX is part of scientific credibility.

---

### Rule 95

Reliability beats novelty.

---

### Rule 96

Performance should feel effortless.

---

### Rule 97

Explainability is mandatory.

---

### Rule 98

The platform should educate—not merely analyze.

---

### Rule 99

Build for the next decade, not the next sprint.

---

### Rule 100

**\*\*Every engineering decision should answer one question:\*\***

> **\*\*"Will this make SwimAnalyzer AI a more trustworthy scientific platform five years from now?"**

If the answer is **\*\*No\*\***, reconsider the decision.

---

# 19.12 Final Engineering Statement

SwimAnalyzer AI is more than a software project.

It is a long-term scientific engineering platform.

These Golden Rules exist to preserve:

- Scientific credibility
- Engineering excellence
- Software quality
- Maintainability
- Scalability
- Trust

Every contributor, whether human or AI, is expected to uphold these principles.

---

## End of Chapter 19

This chapter establishes the permanent engineering philosophy of SwimAnalyzer AI.

These **\*\*100\*\*** Golden Rules serve as the project's constitution, guiding every architectural decision.

> **\*\*Final Principle\*\*** > > **\*\*A great engineering project is not remembered because it was completed.**

---

# 20.1 Purpose

This appendix consolidates the standards, references, diagrams, terminology, conventions, and supporting materials.

It serves as the permanent reference section of the Engineering Playbook.

---

# Appendix A – Project Glossary

| Term          | Definition                                                    |
|---------------|---------------------------------------------------------------|
| -----         | -----                                                         |
| Athlete       | A swimmer whose performance is analyzed.                      |
| Session       | One complete video analysis.                                  |
| Stroke Cycle  | One complete arm movement sequence.                           |
| Stroke Rate   | Number of completed stroke cycles per minute (SPM).           |
| Stroke Length | Estimated distance covered per stroke cycle.                  |
| Body Roll     | Rotation of the swimmer's torso around the longitudinal axis. |

```
| VQA | Video Quality Assessment. |
| Confidence Score | Reliability of the scientific measurements. |
| Performance Score | Overall technique quality score. |
| Landmark | A body keypoint detected by MediaPipe Pose. |
| Pose Visibility | Confidence that a landmark is visible. |
| Calibration | Conversion from image space to meaningful measurements. |
```

```

```

```
Appendix B – Official Folder Structure
```

```
``` SwimAnalyzer_AI/
```

```
| └─ app/ | └─ analysis/ | └─ services/ | └─ repositories/ | └─ database/ | └─ models/
```

```
No new top-level folders should be introduced without architectural review.
```

```
---
```

```
# Appendix C – Naming Convention
```

```
## Files
```

```
``` snake_case.py ```
```

```
Example
```

```
``` analysis_service.py
```

```
pose_detector.py
```

```
video_quality_assessor.py ```
```

```
---
```

```
## Classes
```

```
``` PascalCase ```
```

```
Example
```

```
``` AnalysisService
```

```
PoseDetector
```

```
AthleteRepository ```
```

```
---
```

```
## Functions
```

```
''' snake_case() '''
```

Example

```
''' calculate_score()
```

```
estimate_body_roll() '''
```

```
---
```

```
## Constants
```

```
''' UPPER_CASE '''
```

Example

```
''' DEFAULT_FRAME_RATE
```

```
MAX_UPLOAD_SIZE '''
```

```
---
```

```
# Appendix D – Engineering Diagrams
```

The official project documentation should maintain the following diagrams.

```
## System Architecture
```

```
''' UI
```

```
↓
```

```
### Application Service
```

```
↓
```

```
### Analysis Engine
```

```
↓
```

```
### Scientific Validation
```

```
↓
```

```
Persistence
```

```
↓
```

```
Database '''
```

```
---
```

Scientific Pipeline

``` Video

↓

\*\*VQA\*\*

↓

### Pose Detection

↓

### Stroke Detection

↓

Biomechanics

↓

### Consistency Validation

↓

### Performance Score

↓

Report ```

---

## Database Flow

``` Athlete

↓

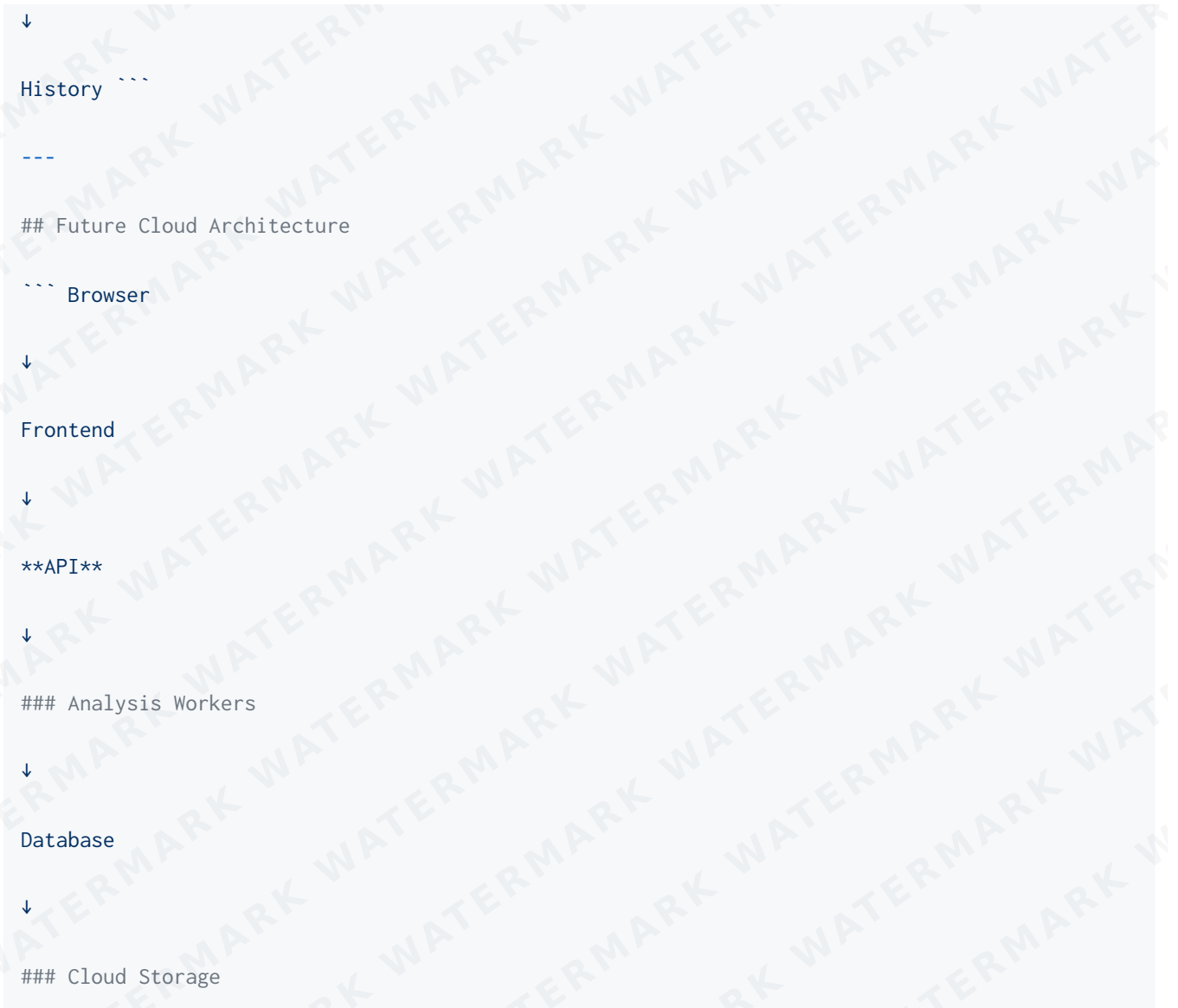
Analysis Session

↓

Reports

↓

Charts



Appendix E — Official Technology Stack

Current

Python

Streamlit

SQLite

MediaPipe

Plotly

FFmpeg

SQLAlchemy

Future

React

FastAPI

PostgreSQL

Redis

Docker

Kubernetes

AWS

Azure

Google Cloud

Appendix F — Scientific References

The scientific implementation should be validated against peer-reviewed literature whenever possible.

Recommended reference categories:

- Swimming biomechanics
- Sports biomechanics
- Human motion analysis
- Computer vision
- Pose estimation
- Machine learning
- Sports performance analysis

Primary reference organizations include:

- World Aquatics
- International Society of Biomechanics

- American College of Sports Medicine (**ACSM**)
 - **IEEE**
 - Springer Nature
 - Elsevier
 - PubMed
-

Appendix G — Coding Standards Summary

The project officially adopts:

- ✓ **PEP-8**
 - ✓ **SOLID**
 - ✓ Repository Pattern
 - ✓ Strategy Pattern
 - ✓ Dependency Injection
 - ✓ Service Layer
 - ✓ Type Hints
 - ✓ Structured Logging
 - ✓ Configuration-Driven Design
-

Appendix H — Quality Standards

Minimum engineering requirements:

Test Coverage

≥ 90%

Architecture Violations

0

Critical Bugs

0

Scientific Contradictions

0

Production Crashes

0

Appendix I – AI Usage Policy

AI may assist with:

Documentation

Refactoring

Testing

Architecture

Code generation

Research summaries

AI must never:

Invent scientific results

Change validated algorithms without review

Generate production code without verification

Replace engineering judgment

Appendix J – Release Policy

Every release must include:

Version number

Release notes

Migration notes

Scientific changes

Performance changes

Known limitations

Rollback procedure

Appendix K — Future Research Topics

Potential future research areas include:

- 3D underwater pose estimation
 - Multi-camera fusion
 - Markerless motion capture
 - Automatic swimmer identification
 - AI-generated coaching plans
 - Injury risk prediction
 - Hydrodynamic efficiency estimation
 - Real-time competition analysis
 - Wearable sensor fusion
 - Foundation models for sports biomechanics
-

Appendix L — Recommended Reading

Software Engineering

- Clean Architecture — Robert C. Martin
- Clean Code — Robert C. Martin
- Designing Data-Intensive Applications — Martin Kleppmann
- The Pragmatic Programmer — Hunt & Thomas
- Domain-Driven Design — Eric Evans

Artificial Intelligence

- Deep Learning — Goodfellow, Bengio & Courville

- Pattern Recognition and Machine Learning — Christopher Bishop
- Hands-On Machine Learning — Aurélien Géron

Computer Vision

- Multiple View Geometry — Hartley & Zisserman
- Computer Vision: Algorithms and Applications — Richard Szeliski

Sports Science

- Biomechanics and Motor Control of Human Movement — David Winter
- Biomechanics of Sport and Exercise — McGinnis

Appendix M — Engineering Philosophy

SwimAnalyzer AI is designed according to four permanent pillars.

``` ### Scientific Integrity

↓

## Engineering Excellence

↓

## Artificial Intelligence

↓

Long-Term Maintainability ```

Removing any pillar weakens the entire platform.

---

# Appendix N — Official Project Statement

---

SwimAnalyzer AI is engineered as a scientific software platform whose primary objective is to provide reliable, explainable, and scalable swimming performance analysis.

Every engineering decision should strengthen:

- Scientific credibility
- Software quality
- User trust
- Long-term sustainability

The platform should remain understandable, extensible, and professionally maintainable for many years, regardless of changes in technology, AI models, or engineering teams.

---

# End of Engineering Playbook

---

## Document Information

---

### Title

SwimAnalyzer AI Engineering Playbook

### Version

1.0.0

### Status

## Official Engineering Reference

### Maintained By

SwimAnalyzer AI Engineering Team

### Intended Audience

- Software Engineers
- AI Engineers
- Sports Scientists
- Biomechanics Researchers
- QA Engineers
- DevOps Engineers
- Technical Product Managers
- Future Contributors

---

## Final Engineering Statement

---

**"Build software that scientists can trust, engineers can maintain, coaches can rely on, and athletes can improve because of."**

This principle defines the long-term mission of SwimAnalyzer AI and serves as the foundation for every future release. ````